答案速查

数据结构

题号	答案	正文页	解析页
1-1	C	1	6
1-2	B	3	6
1-3	B	3	6
1-4	B	3	6
2-1	D	5	6
2-2	双指针，时间 $O(n)$，空间 $O(1)$	8	6
2-3	D	9	7
2-4	三次逆置，时间 $O(n)$，空间 $O(1)$	10	7
2-5	对齐表尾后同步比较地址，时间 $O(m+n)$	10	7
2-6	抵消候选后再次计数，时间 $O(n)$，空间 $O(1)$	11	8
3-1	C	12	8
3-2	C	14	9
3-3	B	18	9
3-4	C	18	10
3-5	B	16	10
4-1	位置均为 4；朴素 15 次，普通 KMP10 次	22	10
4-2	位置 5；普通 next 比较 12 次，nextval 比较 9 次	23	10
5-1	B	24	11
5-2	A	25	11
5-3	A	27	11
5-4	B	30	12
5-5	B	30	13
5-6	C	30	13
6-1	C	33	13
6-2	D	35	14
6-3	C	36	14
6-4	C	37	14
6-5	每轮零入度候选恰有一个，且处理完全体顶点	38	14
6-6	C	39	15
7-1	C	40	15
7-2	B	41	15
7-3	最终为 34(23(15,28),107(98,115))	44	16
7-4	D	44	16
7-5	A、D	46	16
7-6	C	46	16
7-7	C	47	17
7-8	HT=[11, 空,14,7, 空,20,9, 空, 空,3,18]；7/11；3、18、14；地址 7	47	17
8-1	B	51	17
8-2	D	52	17
8-3	C	53	17
8-4	C	53	18
8-5	C	54	18
8-6	3 段；最大 n，最小 m	55	18
8-7	825 次；反复合并当前两个最短表	56	19
8-8	WPL=690	56	19
8-9	随机划分选择第 10 小，平均时间 $O(n)$、迭代辅助空间 $O(1)$	52	19

计算机组成原理

题号	答案	正文页	解析页
1-1	A	60	20
1-2	A	61	20
1-3	D	61	20
2-1	B	63	20
2-2	B	64	21
2-3	D	64	21
2-4	B	66	21
2-5	C	66	21
2-6	B	67	22
2-7	A	69	22
2-8	A	70	22
2-9	D	70	22
2-10	（1）机器模型下循环条件恒真；改 int 后不进入循环。（2）相等；00FF FFFFH、4B7F FFFFH。（3）浮点舍入为 2^25。（4）低 32 位全 1；最大 n=30。（5）正无穷；不溢出最大 n=126，精确最大 n=23。	71	22
2-11	D	72	23
3-1	A	74	23
3-2	D	74	23
3-3	D	75	23
3-4	D	75	24
3-5	B	77	24
3-6	C	78	24
3-7	A	79	24
3-8	C	79	25
3-9	C	81	25
3-10	C	81	25
3-11	（1）2 页、2 次缺页，0042 2000H 与 0042 3000H。（2）以数组元素计，无时间局部性，有空间局部性。（3）块内 A4～A0，组号 A10～A5；a[1][0]=0042 2100H，组号 8。（4）192 块；写分配下两种循环均 87.5%。	82	25
3-12	（1）VPN=A29～A12，偏移 =A11～A0。（2）标记 =A29～A15，组号 =A14～A12。（3）淘汰 VPN=4。（4）每表项增加 2 位。	83	26
3-13	（1）虚址 6/2/6 位，物理址 6/4/2 位。（2）03D4H 读得 36H；00F1H 从主存取数，字节值未知；0229H 缺页。	83	26
4-1	D	85	27
4-2	C	86	27
4-3	C	86	27
4-4	A	87	27
4-5	（1）10 次，第 16 行；（2）第 12 行条件转移，第 16、20、30 行无条件改变后继；（3）0040102AH，位移 –42，小端；（4）32 位溢出，改用 long long；（5）高 32 位不等于低 32 位的符号扩展时 OF=1，增加溢出自陷指令。	89	27
5-1	B	91	28
5-2	（1）SF=F15，OF 见解析；（2）Y 保存首操作数，Z 暂存结果；（3）16 个，来自 IR，rd 接写地址译码器；（4）PCout/-MARin→Read→MDRout/IRin，至少 7 拍；（5）CU 产生，IR 和 FR 提供输入。	92	28
5-3	B	93	28
5-4	C	94	29
5-5	C	94	29
5-6	B	94	29

题号	答案	正文页	解析页
5-7	A	95	29
5-8	D	96	29
5-9	C	96	30
5-10	C	97	30
5-11	（1）FEFFH；（2）8 拍；（3）I3 等待 RAW 写回，I4 受前级冻结阻塞；（4）至少 17 拍。	98	30
6-1	B	101	31
6-2	10.664 GB/s；改给 333.25 MHz 且每拍 4 次时相同。	101	31
6-3	D	101	31
6-4	B	101	31
7-1	D	104	32
7-2	C	105	32
7-3	A、B、C 屏蔽字依次 100、110、111；开始 A→B→C，完成 C→B→A。	106	32
7-4	B	106	32
7-5	C	107	32
7-6	D	107	33
7-7	（1）最多 2 μs 查询一次，至少 4%；（2）不能，需 800% 的 CPU 预算；（3）最多 4%。	108	33

操作系统

题号	答案	正文页	解析页
1-1	D	109	34
1-2	B	111	34
1-3	D	112	34
1-4	A	112	34
2-1	C	115	35
2-2	D	117	35
2-3	B	117	35
2-4	C	117	35
2-5	C	118	36
2-6	SRTF：13；RR：19.25	118	36
2-7	$priority = nice + k_1 cpuTime - k_2 waitTime$，$k_1, k_2 > 0$	118	37
2-8	C	119	37
2-9	D	120	37
2-10	两个信号量，初值均 0	120	37
2-11	（1）是；（2）done=0；（3）mutex=1	121	37
2-12	empty=N，odd=even=0，mutex=1；按类别发布	122	38
2-13	empty=1000，full=0，mutex=batch=1	122	38
2-14	D	124	38
2-15	B	125	39
2-16	D	125	39
2-17	C；仅 I	126	39
3-1	C	127	40
3-2	B	128	40
3-3	B	129	40
3-4	A	129	40
3-5	C	131	40
3-6	C；60A01H	132	41
3-7	A	132	41

题号	答案	正文页	解析页
3-8	210 ns；100000220 ns；110 ns；101565H	133	41
3-9	10801008H；042H、001H；00201108H；00301004H	133	41
4-1	B	135	42
4-2	D	136	43
4-3	B	137	43
4-4	D	137	43
4-5	B	138	44
4-6	C	139	44
4-7	（course,2）、（doc,10）；30；2 块；一级和二级	140	44
4-8	$(8+1024+1024^2+1024^3)4096$ B；2^{26} 个；时间不同	141	44
5-1	C	143	44
5-2	B	144	44
5-3	B	145	45
5-4	B	146	45
5-5	C	146	45
5-6	②→⑥→④→③→①→⑤；③ 为驱动；处理入口时 P 阻塞、CPU 内核态	147	45
5-7	307200000 B；100260→101660→110560→60005；(100,5,60)；驱动	147	45

计算机网络

题号	答案	正文页	解析页
1-1	B	150	46
1-2	B	150	46
1-3	D	150	46
1-4	B	151	46
2-1	C	152	47
2-2	A	152	47
2-3	C	154	47
3-1	A	155	47
3-2	D	156	48
3-3	C	156	48
3-4	D	157	48
3-5	C	157	48
3-6	C	157	48
3-7	B	159	49
3-8	D	160	49
3-9	B	160	49
3-10	A	161	49
3-11	B	162	49
3-12	（1）3 帧；（2）5 帧，首 $S_{5,2}$、末 $S_{1,2}$；（3）3 帧，首 $S_{2,3}$；（4）50%。	157	50
4-1	D	163	50
4-2	B	165	50
4-3	B	165	50
4-4	B	166	51
4-5	C	166	51
4-6	B	167	51
4-7	C	168	52
4-8	C	168	52

题号	答案	正文页	解析页
4-9	D	169	52
4-10	（1）192.168.1.127，192.168.1.128/25，45 台；（2）2 片，796 B 与 724 B。	171	52
4-11	3 条路由；目的 194.17.20.200 走 E0；域间 BGP，由 TCP 承载。	166	52
4-12	（1）OSPF；（2）16；（3）30 s、60 s；（4）eBGP/UPDATE、iBGP；（5）R14 选 R11，R15 选 R13。	170	53
4-13	（1）0.24 s、200 kb/s、理想下界 0.40 s；（2）8 帧、4 bit；（3）作业区 10.10.10.64/26，管理区 10.10.10.0/26，生活区 10.10.10.128/25。	172	53
5-1	B	173	54
5-2	C	174	54
5-3	D	175	54
5-4	D	176	55
5-5	D	178	55
5-6	A	177	55
5-7	A	178	55
5-8	（1）H 发送 1、3、4；握手 1、2、3；填充 3、5；（2）16 B；（3）15 个路由器。	178	55
5-9	（1）SYN=ACK=1，ack=101；（2）rwnd=12 KB，cwnd=9 KB，发送窗口 9 KB；（3）20581，163.84 kb/s；（4）300 ms。	179	56
6-1	C	182	57
6-2	A	182	57
6-3	A	183	57
6-4	D	184	57
6-5	A	184	58
6-6	D	185	58
6-7	B	185	58
6-8	（1）控制持久、数据非持久，先建控制；（2）101、18102；（3）3 MSS、5 MSS；（4）60 ms、2.4 Mb/s。	185	59
6-9	（1）DNS，UDP/IP/以太网封装；（2）H1→4、DNS→1、R→2；（3）2 帧，目的全 FF（需新 DNS 查询）。	186	59

1 数据结构

绪论

例 1-1 C

05. C
在存储数据时，不仅要存储数据元素的值，而且要存储数据元素之间的关系。

例 1-2 B

执行 k 轮后，$sum = k(k+1)/2$。终止轮数为满足 $k(k+1)/2 \geq n$ 的最小整数，故 $k = \Theta(\sqrt{n})$；每轮常数时间，选 B。

例 1-3 B

各轮内循环次数为 $1, 2, 4, \ldots, 2^k$，其中 $2^k < n \leq 2^{k+1}$。总次数 $2^{k+1} - 1 = \Theta(n)$，选 B。

例 1-4 B

16. B
假设第 k 次循环终止，则第 k 次执行时，$(x+1)^2 > n$，x 的初始值为 0，第 k 次判断时，$x = k-1$，即 $k^2 > n$，$k > \sqrt{n}$，因此该程序段的时间复杂度为 $O(n^{1/2})$。

线性表

例 2-1 D

12. D
线性表有顺序存储和链式存储两种存储结构。若采用链式存储结构，则删除元素 a_{50} 不需要移动元素；若采用顺序存储结构，则需要依次移动 50 个元素。

例 2-2 双指针，时间 $O(n)$，空间 $O(1)$

倒数位置由尾部决定，但链表不能直接向前访问。让两个指针保持固定间隔：`fast` 先从首元结点前进 k 步，再让 `fast` 与 `slow` 同步前进。若表长为 n，`fast` 到达空位置时共走了 n 步，`slow` 只走了 $n-k$ 步，正好指向第 $n-k+1$ 个结点。

```
int searchK(LNode *list, int k) {
    if (k<=0) return 0;
    LNode *fast=list->next, *slow=list->next;
    for (int j=0; j<k; ++j) {
        if (fast==NULL) return 0;   // 表长不足 k
        fast=fast->next;
    }
    while (fast!=NULL) {
        fast=fast->next;
        slow=slow->next;
    }
    printf("%d", slow->data);
    return 1;
}
```

以长度 5、$k=2$ 为例，先让 `fast` 指向第 3 个结点，`slow` 留在第 1 个；同步经过 “4 与 2—5 与 3—空与 4”，第 4 个即倒数第 2 个。$k=n$ 时第一阶段恰好走到空位置，第二阶段不执行，返回首元结点；$k>n$ 时在第一阶段报失败。两个指针合计走 $O(n)$ 步，辅助空间 $O(1)$，没有修改任何链接。

例 2-3　D

15．D

为了在 p 之前插入结点 q，可以将 p 的前一个结点的 next 域指向 q，将 q 的 next 域指向 p，将 q 的 prior 域指向 p 的前一个结点，将 p 的 prior 域指向 q。仅选项 D 满足条件。

例 2-4　三次逆置，时间 $O(n)$，空间 $O(1)$

题目要求两个连续区段交换位置，同时保持各段内部次序。设前 p 个元素为 A，其余为 B。分别逆置得 A^RB^R，再整体逆置，即得 BA。两个区段各被逆置两次，内部方向恢复，而相对位置交换。

```
void reverse(int R[], int l, int r) {
    while (l<r) {
        int t=R[l]; R[l]=R[r]; R[r]=t;
        ++l; --r;
    }
}
void rotateLeft(int R[], int n, int p) {
    reverse(R,0,p-1);
    reverse(R,p,n-1);
    reverse(R,0,n-1);
}
```

例如 `abcdefgh` 左移 3 位，依次得到 `cbadefgh`、`cbahgfed`、`defghabc`。总交换次数不超过 n，时间 $O(n)$；只用常数临时变量，空间 $O(1)$。若将首元素移到末尾重复 p 次，时间为 $O(np)$，当 p 与 n 同阶时会退化。三次逆置可处理 $p=1$ 和 $p=n-1$，无需另写搬移循环。

例 2-5　对齐表尾后同步比较地址，时间 $O(m+n)$

共享指的是地址相同，不能以 `p->data==q->data` 判断。单链表一旦在某结点汇合，其后继也相同，所以两表从公共起点到末尾的长度相等。先各扫描一次求长 m,n，让较长表先走 $|m-n|$ 步，两指针便处于距表尾相同的位置；此后同步前进，第一次地址相等的位置就是答案。

```
LNode *commonSuffix(LNode *str1, LNode *str2) {
    int m=0,n=0;
    for (LNode *p=str1->next; p; p=p->next) ++m;
    for (LNode *q=str2->next; q; q=q->next) ++n;
    LNode *p=str1->next, *q=str2->next;
    while (m>n) { p=p->next; --m; }
    while (n>m) { q=q->next; --n; }
    while (p!=q) { p=p->next; q=q->next; }
    return p;                    // 无公共结点时二者同时为 NULL
}
```

图中两表长度分别为 7 和 5，先让长表跨过 `l,o`，再比较 `a` 与 `b`、`d` 与 `e`，最后到达同一个 `i`。若两表内容相同却各自分配结点，则直到空位置才相等，返回空。总时间 $O(m+n)$，辅助空间 $O(1)$，适用于有限、无环单链表。

例 2-6　抵消候选后再次计数，时间 $O(n)$，空间 $O(1)$

每次删除两个值不同的元素，不会消灭一个原本占严格多数的值：若删去一个多数元素和一个其他元素，多数仍超过剩余的一半；若删去两个其他元素，多数优势更大。用一个候选值和计数模拟这种抵消。计数为 0 时选新候选；相同则加 1，不同则减 1。

```
int majority(int A[], int n) {
    int candidate=0, count=0;
    for (int i=0; i<n; ++i) {
        if (count==0) { candidate=A[i]; count=1; }
        else if (A[i]==candidate) ++count;
        else --count;
    }
    int actual=0;
    for (int i=0; i<n; ++i)
        if (A[i]==candidate) ++actual;
    return actual>n/2 ? candidate : -1;
}
```

抵消只保证“若有主元素，最后候选一定是它”，没有保证主元素存在。例如 `1,2,3` 最后仍可能留下候选 3，它只出现一次，所以必须第二次扫描核实。两个线性扫描合计 $O(n)$ 时间，额外空间 $O(1)$。题目给出的值域也允许用计数数组，但需 $O(n)$ 辅助空间；此处没有必要为省下一次扫描而付出这部分空间。

栈、队列和数组

例 3-1　C

26. C

时刻注意栈的特点是先进后出，下表是出入栈的详细过程。

序号	说明	栈内	栈外	序号	说明	栈内	栈外
1	*a* 入栈	*a*		8	*e* 入栈	*ae*	*bdc*
2	*b* 入栈	*ab*		9	*f* 入栈	*aef*	*bdc*
3	*b* 出栈	*a*	*b*	10	*f* 出栈	*ae*	*bdcf*
4	*c* 入栈	*ac*	*b*	11	*e* 出栈	*a*	*bdcfe*
5	*d* 入栈	*acd*	*b*	12	*a* 出栈		*bdcfea*
6	*d* 出栈	*ac*	*bd*	13	*g* 入栈	*g*	*bdcfea*
7	*c* 出栈	*a*	*bdc*	14	*g* 出栈		*bdcfeag*

栈内的最大深度为 3，所以栈 S 的容量至少是 3。

【另解】因为元素的出队顺序和入队顺序相同，所以元素的出栈顺序就是 *b*, *d*, *c*, *f*, *e*, *a*, *g*，因此元素的出入栈次序为 Push(S, *a*), Push(S, *b*), Pop(S, *b*), Push(S, *c*), Push(S, *d*), Pop(S, *d*), Pop(S, *c*), Push(S, *e*), Push(S, *f*), Pop(S, *f*), Pop(S, *e*), Pop(S, *a*), Push(S, *g*), Pop(S, *g*)。初始所需容量为 0，每做一次 Push 操作，容量加 1；每做一次 Pop 操作，容量减 1，记录的容量最大值为 3。

例 3-2 C

06. C

队列的长度为(rear-front+maxsize)%maxsize=(rear-front+21)%21=16。这种情况和 front 指向当前元素，rear 指向队尾元素的下一个元素的计算是相同的。

注 意

数组 A[n]的下标范围为 0～$n-1$。若写成 A[0…n]，则说明下标范围为 0～n。

例 3-3 B

11. B

三对角矩阵如下所示。

$$\begin{bmatrix} a_{1,1} & a_{1,2} & & & & \\ a_{2,1} & a_{2,2} & a_{2,3} & & 0 & \\ & a_{3,2} & a_{3,3} & a_{3,4} & & \\ & & \ddots & \ddots & \ddots & \\ & 0 & & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\ & & & & a_{n,n-1} & a_{n,n} \end{bmatrix}$$

采用压缩存储，将 3 条对角线上的元素按行优先方式存放在一维数组 B 中，且 $a_{1,1}$ 存放于 B[0]中，其存储形式如下所示：

$a_{1,1}$	$a_{1,2}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	…	$a_{n-1,n}$	$a_{n,n-1}$	$a_{n,n}$

可以计算矩阵 $\boldsymbol{A}$ 中 3 条对角线上的元素 $a_{i,j}$（$1 \leqslant i,j \leqslant n, |i-j| \leqslant 1$）在一维数组 B 中存放的下标为 $k=2i+j-3$，公式很难记忆，我们通常采用解法 2。

解法 1：针对该题，仅需将数字逐一代入公式：$k=2\times30+30-3=87$，结果为 87。

解法 2：观察上图的三对角矩阵不难发现，第一行有两个元素，剩下的在元素 $m_{30,30}$ 所在行之前的 28 行（注意下标 $1 \leqslant i,j \leqslant 100$）中，每行都有 3 个元素，而 $m_{30,30}$ 之前仅有一个元素 $m_{30,29}$，不难发现元素 $m_{30,30}$ 在数组 N 中的下标是 $2+28\times3+2-1=87$。

注 意

矩阵和数组的下标从 0 或 1 开始（如矩阵可能从 $a_{0,0}$ 或 $a_{1,1}$ 开始，数组可能从 B[0]或 B[1]开始），这时就需要适时调整计算方法（方法无非是多计算 1 或少计算 1 的问题）。

例 3-4 C

14. C

上三角矩阵按列优先存储，先存储只有 1 个元素的第一列，再存储有 2 个元素的第二列，以此类推。$m_{7,2}$ 位于左下角，对应右上角的元素为 $m_{2,7}$，在 $m_{2,7}$ 之前存有

第 1 列：1

第 2 列：2

……

第 6 列：6

第 7 列：1

前面共存储有 $1+2+3+4+5+6+1=22$ 个元素（数组下标范围为 0～21），注意数组下标从 0 开始，所以 $m_{2,7}$ 在数组 N 中的下标为 22，即 $m_{7,2}$ 在数组 N 中的下标为 22。

例 3-5 B

18. B

第一次调用：① 从 S1 中弹出 2 和 3；② 从 S2 中弹出+；③ 执行 3+2=5；④ 将 5 压入 S1 中，第一次调用结束后 S1 中剩余 5、8、5（5 在栈顶），S2 中剩余*、-（-在栈顶）。第二次调用：① 从 S1 中弹出 5 和 8；② 从 S2 中弹出-；③ 执行 8-5=3；④ 将 3 压入 S1 中，第二次调用结束后 S1 中剩余 5、3（3 在栈顶），S2 中剩余*。第三次调用：① 从 S1 中弹出 3 和 5；② 从 S2 中弹出*；③ 执行 5*3=15；④ 将 15 压入 S1 中，第三次调用结束后 S1 中仅剩余 15，S2 为空。

串

例 4-1 位置均为 4；朴素 15 次，普通 KMP10 次

朴素匹配从起点 1、2、3、4 依次比较 6、1、2、6 次，共 15 次。普通 next 为 `0,1,1,2,2,3`；KMP 前 5 对相等，第 6 对在 `(i,j)=(6,6)` 失配，令 `j=3`、`i` 保持 6；随后 `(6,3)`、`(7,4)`、`(8,5)`、`(9,6)` 均相等，共 10 次字符比较，返回 `10-6=4`。

例 4-2 位置 5；普通 next 比较 12 次，nextval 比较 9 次

普通 next 为 `0,1,2,3,4`，nextval 为 `0,0,0,0,4`。前 3 对 `a` 相等；主串第 4 位 `c` 在普通表下依次与模式第 4、3、2、1 位失配，优化表在第一次失配后直接退到 0。推进到主串第 5 位后，连续匹配 `aaaab`。普通表比较 `3+4+5=12` 次，优化表比较 `3+1+5=9` 次，均返回 5。

树与二叉树

例 5-1 B

09. B

设树中度为 i（$i=0, 1, 2, 3, 4$）的结点数分别为 n_i，树中结点总数为 n，则 $n=$ 分支数 $+1$，而分支数又等于树中各结点的度之和，即 $n=1+n_1+2n_2+3n_3+4n_4=n_0+n_1+n_2+n_3+n_4$。依题意，$n_1+2n_2+3n_3+4n_4=10+2+30+80=122$，$n_1+n_2+n_3+n_4=10+1+10+20=41$，可得出 $n_0=82$，即树 T 的叶结点的个数是 82。

例 5-2 A

28. A

二叉树采用顺序存储时，用数组下标来表示结点之间的父子关系。对于一棵高度为 5 的二叉树，为了满足任意性，其 1～5 层的所有结点都要被存储起来，即考虑为一棵高度为 5 的满二叉树，共需要存储单元的数量为 $1+2+4+8+16=31$。

例 5-3 A

43. A

根据中序遍历的特点，v 有左右子树，以 v 为根的子树的中序序列为：v 的左子树的中序序列, v, v 的右子树的中序序列。又因为 T 的中序序列为“…, p, v, q,…”，可知 p 属于 v 的左子树，q 属于 v 的右子树。在 v 的左子树的中序序列中：假设 p 有右孩子，则 p 的右孩子在中序序列中应在 p 之后，与 p 是最后一个遍历结点矛盾，因此 p 不存在右孩子。在 v 的右子树的中序序列中：假设 q 存在左孩子，则 q 的左孩子在中序序列中应在 q 之前，与 q 是第一个遍历结点矛盾，因此 q 不存在左孩子。

例 5-4 B

15. B

森林与二叉树的转换规则为“左孩子右兄弟”。在最后生成的二叉树中，父子关系在对应森林关系中可能是兄弟关系或者原本就是父子关系。

说法 I：若结点 v 是结点 u 的第二个孩子结点，转换时，结点 v 就变成结点 u 第一个孩子的右孩子，符合要求。说法 II：结点 u 和 v 是兄弟结点的关系，但二者之中还有一个兄弟结点 k，则转换后结点 v 就变为结点 k 的右孩子，而结点 k 则是结点 u 的右孩子，符合要求。

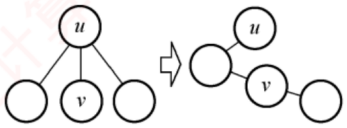


图 I

图 II

说法 III：若结点 u 的父结点与 v 的父结点是兄弟关系，则转换后，结点 u 和 v 分别在两者最左父结点的两棵子树中，不可能出现在同一条路径中。

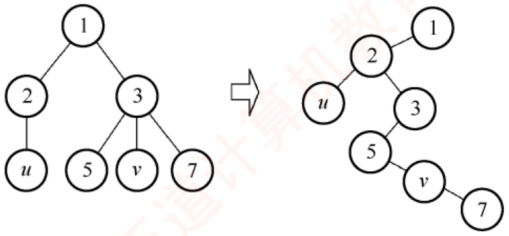
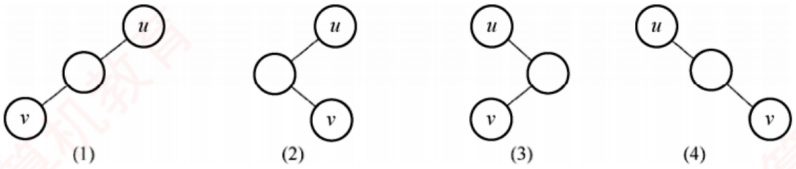


图 III

【另解】由题意可知 u 是 v 的父结点的父结点，如下图所示，有四种情况：



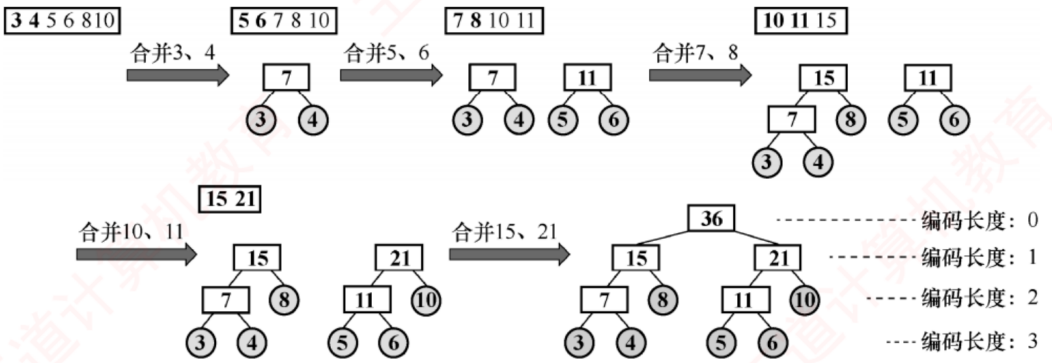
(1) (2) (3) (4)

根据树与二叉树的转换规则，将这四种情况转换成树中结点的关系。（1）在原来的树中 u 是 v 的父结点的父结点；（2）在树中 u 是 v 的父结点；（3）在树中 u 是 v 的父结点的兄弟；（4）在树中 u 与 v 是兄弟关系。由此可知说法 I 和 II 正确。

例 5-5 B

22. B

构建哈夫曼树的过程如下图所示。

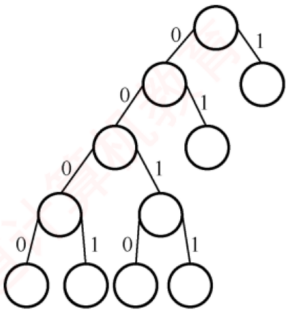


对叶结点的哈夫曼编码，共有 4 个长度为 3 的叶结点、2 个长度为 2 的叶结点，编码的加权平均长度为[(3+4+5+6)×3+(8+10)×2]/(3+4+5+6+8+10)=2.5。

例 5-6 C

04. C

在哈夫曼编码中，一个编码不能是任何其他编码的前缀。3 位编码可能是 001，对应的 4 位编码只能是 0000 和 0001。3 位编码也可能是 000，对应的 4 位编码只能是 0010 和 0011。若全采用 4 位编码，则可以为 0000, 0001, 0010 和 0011。题中问的是最多，所以选择选项 C。



【另解】若哈夫曼编码的长度只允许小于或等于 4，则哈夫曼树的高度最高是 5，已知一个字符编码为 1，另一个字符编码为 01，这说明第二层和第三层各有一个叶结点，为使得该树从第 3 层起能够对尽可能多的字符编码，余下的二叉树应该是满二叉树，如右图所示，底层可以有 4 个叶结点，最多可以再对 4 个字符编码。

图

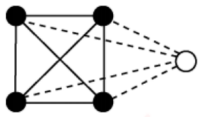
例 6-1 C

16. C

题干要求无论如何分配边，都能使 7 个顶点连通，这不同于只要 6 条边两两相连就能构成一

个连通图的情形。考虑最极端的情形，即图 G 的某 6 个顶点构成一个完全无向图，此时若再添加一条边，则都将连通第 7 个顶点，使该图变成一个连通图。所以最少边数 = 6×5/2 + 1 = 16。若边数 n 小于或等于 15，可以使这 n 条边仅连接图 G 中的某 6 个顶点，从而导致第 7 个顶点无法与这 6 个顶点构成连通图（不满足“在任何情况下”）。

为简单起见，以 5 个顶点为例，左边 4 个顶点和 4×3/2 = 6 条边构成一个完全图，此时若再添加一条边（可以是虚线中的任意一条），则能保证这 5 个顶点在任何情况下都是连通的，如右图所示。若边数小于 7，则不能保证 5 个顶点在任何情况下都是连通的。



例 6-2 D

11. D

画出 V 和 E 对应的图 G，然后根据搜索算法求解。

注 意

为什么本题序列是不唯一的，而上题序列却是唯一的呢？

因为上题给出了具体的存储结构，此时就必须按照算法的过程来执行，每个顶点的邻接点的顺序已固定，但本题中每个顶点的邻接点的顺序是非固定的。

例 6-3 C

两者首先选权 5 的 (V_1, V_4)。Kruskal 第 2 条可取不成环的任一权 8 边，包括 (V_2, V_3)。Prim 此时集合为 $\{V_1, V_4\}$，只能选跨割边 (V_1, V_3) 或 (V_4, V_3)，选 C。

例 6-4 C

起点确定后初始向量为 $(26, 3, \infty, 6)$。先确定 3，经 3 到 2 得到 $3 + 22 = 25$；再确定 5，经 5 到 2 得到 $6 + 15 = 21$，经 5 到 4 得到 $6 + 8 = 14$，故为 $(21, 3, 14, 6)$，选 C。经 4 把到 2 改成 15 属于下一轮。

例 6-5 每轮零入度候选恰有一个，且处理完全体顶点

13. 【解答】

1）算法的基本设计思想：建立图 G 各顶点的入度表 degree[]。选择入度为 0 的顶点 v，将 v 的所有邻接点的入度减 1，重复执行这个过程。若每次选中的入度为 0 的顶点有且仅有一个，且共进行了 G.numVertices 次，则意味着存在唯一的拓扑序列，返回 1，否则不存在拓扑序列，或者存在多个拓扑序列，返回 0。

2）用 C 语言描述的算法如下：

```
int uniquely(MGraph G){//判定有向图是否存在唯一的拓扑序列
    int *degree,i,j,count=0,in0=-1,prev_in0;
    degree=(int*)malloc(G.numVertices*sizeof(int));
    for(j=0;j<G.numVertices;j++){//计算各顶点的入度
        degree[j]=0;
        for(i=0;i<G.numVertices;i++)
            degree[j]+=G.Edge[i][j];
        if(degree[j]==0){
```

```
                if(in0==-1) in0=j;          //入度为 0 的顶点
                else in0=-2;                //有多个入度为 0 的顶点
            }
        }
        while(in0>=0){
            count++;
            prev_in0=in0;
            in0=-1;
            for(j=0;j<G.numVertices;j++)
                if(G.Edge[prev_in0][j]>0){
                    if(--degree[j]==0){     //邻接点入度值减 1
                        if(in0==-1) in0=j; //入度为 0 的顶点
                        else in0=-2;        //有多个入度为 0 的顶点
                    }
                }
        }
        free(degree);
        if(count==G.numVertices) return 1;
        else return 0;
    }
```

例 6-6 C

按拓扑序 1、3、2、4、5、6 求得事件最早、最迟时间均依次为 0、8、12、21、18、27。工期 27，关键路径为 b-d-c-g、b-d-e-h、b-f-h。同时缩短 d、f 可覆盖三条关键路径；A、B 遗漏 b-f-h，D 遗漏 b-d-c-g，选 C。持续缩短后应重算新关键路径。

查找

例 7-1 C

26. C

本题可直接套用书中的结论“每块记录数=$\sqrt{n}$ 时，平均查找长度最小”。解释如下：分块查找的平均查找长度（ASL）等于查找索引表的 ASL 与块内查找的 ASL 之和。设每块含 s 个元素，则块数为 $k=400/s$。总 ASL $=(k+1)/2+(s+1)/2=(k+s)/2+1$。要使 ASL 最小，需要最小化 $k+s=400/s+s$。根据均值不等式，当 $400/s=s$ 时取最小值，解得 $s=20$。

例 7-2 B

20. B

折半查找法在查找不成功时和给定值进行关键字的比较次数最多为树的高度，即$\lfloor \log_2 n \rfloor + 1$或$\lceil \log_2(n+1) \rceil$。在本题中，$n=16$，所以比较次数最多为 5。

注 意

在折半查找判定树中的方形结点是虚构的，它不计入比较的次数。

例 7-3　最终为 34(23(15,28),107(98,115))

以下用 **根（左子树，右子树）**表示局部结构，⌀ 表示空。

插入	失衡处与调整	调整后整树
34	无	34
23	无	34(23,∅)
15	34 处 LL，右旋	23(15,34)
98	无	23(15,34(∅,98))
115	34 处 RR，左旋	23(15,98(34,115))
28	23 处 RL，先右旋 98，再左旋 23	34(23(15,28),98(∅,115))
107	98 处 RL，先右旋 115，再左旋 98	34(23(15,28),107(98,115))

插入 28 时，34 仍平衡，最小失衡祖先为 23；新增路径为 “23 的右—98 的左”，采用 RL。各步中序均为已插入关键字的升序。

例 7-4　D

29．D

大多数教材将平衡二叉树定义为一种高度平衡的二叉排序树，二叉排序树的中序序列是一个升序序列，而题意正好相反。由此可知，命题老师认为平衡二叉树仅为一棵满足高度平衡的二叉树，不一定是二叉排序树。只有两个结点的平衡二叉树的根结点的度为 1，选项 A 错误。中序遍历后得到一个降序序列（与二叉排序树正好相反），树中最大元素一定无左子树（可能有右子树），这与二叉排序树也正好相反，也因此不一定是叶结点，选项 B 错误，选项 D 正确。最后插入的结点可能会导致平衡调整，而不一定是叶结点，选项 C 错误。

例 7-5　A、D

09．A、D

利用前面的公式即最小高度 $h \geqslant \log_m(n+1)$和最大高度 $H \leqslant \log_{\lceil m/2 \rceil}[(n+1)/2]+1$，易算出最大高度 $H \leqslant \log_2[(2047+1)/2]+1=11$，最小高度 $h \geqslant \log_3 2048=6.9$，从而最小高度取 7（注意，有些辅导书针对本题算出的高度要比这里给出的答案多 1，因为它们在对 B 树的高度定义中，把最底层不包含任何关键字的叶结点也算进去了）。

例 7-6　C

25．C

对于 4 阶 B 树，每个结点包含 1～3 个关键字（根结点可为 1～3 个，非根结点至少 1 个）。给定 7 个不同关键字，按树高分类讨论：① **当树为 2 层时**，若根含 1 个关键字，则其 2 个子结点需要分配 6 个关键字，唯一合法的分配为(3, 3)，对应 1 种；若根含 2 个关键字，则 3 个子结点分配 5 个关键字，合法分配为(3, 1, 1)和(2, 2, 1)，考虑所有排列共 6 种；若根含 3 个关键字，则 4 个子结点各含 1 个关键字，对应 1 种；合计 8 种。② **当树为 3 层时**，仅当根及其两个子结点均含 1 个关键字时，剩余 4 个关键字恰好分配给 4 个叶子结点（各 1 个），满足最小关键字数要求，仅 1 种。综上，共可构造 8 + 1 = 9 种不同的 4 阶 B 树。

例 7-7 C

23. C

当采用开放定址法时，不能随便物理删除表中的已有元素，因为若删除元素，则可能截断其他具有相同散列地址的元素的查找地址。因此，当要删除一个元素时，可给它做一个删除标记。依次将 2022, 12, 25 插入散列表，然后删除 25，得到的散列表如下：

地址	0	1	2	3	4
关键字		2022	12		25（删除）
查找失败次数	1	3	2	1	2

当查找位置是删除标记时，应继续往后查找。

查找失败的平均查找长度为(1 + 3 + 2 + 1 + 2)/5 = 1.8。

例 7-8 HT=[11, 空,14,7, 空,20,9, 空, 空,3,18]；7/11；3、18、14；地址 7

20、3、11、18 直接落 5、9、0、10；9 从 5 到 6；14 从 9 依次探测 10、2；7 从 10 依次探测 0、3。最终 0 至 10 为 `11, 空,14,7, 空,20,9, 空, 空,3,18`，装填因子 7/11。查 14 访问 9、10、2，比较 3、18、14。查 8 初始地址 2，按 $H_0 + k^2$ 探测 2、3、6、0、7，在空地址 7 失败。

排序

例 8-1 B

12. B

第一趟增量 $d = 5$，第一趟排序后，结果为 2, 11, 5, 1, 8, 9, 24, 7, 34, 51, 13, 77, 56。第二趟增量 $d = 3$，第二趟排序后，结果为 1, 7, 5, 2, 8, 9, 24, 11, 34, 51, 13, 77, 56。

例 8-2 D

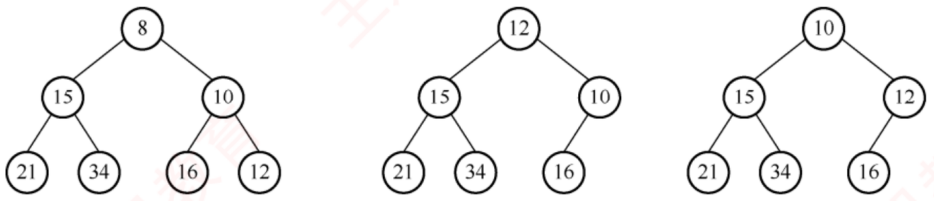
18. D

基于上题中分析得出的结论，观察题中的四个选项，选项 A 的 28, 72 符合第一种情况；选项 B 的 2, 72 符合第一种情况；选项 C 的 2, 28, 32 符合第一种或第二种情况；最后看选项 D，只有 12 和 32 处于最终位置，既不符合第一种情况，又不符合第二种情况。

例 8-3 C

17. C

删除 8 后，将 12 移动到堆顶，第一次是 15 和 10 比较，第二次是 10 和 12 比较并交换，第三次还需比较 12 和 16，所以比较次数为 3。



例 8-4 C

20. C

从左至右对这三个有序序列进行 2 路归并排序的过程如下。

① 合并(3, 5)和(7, 9)：比较 3 和 7（1 次），选择 3；比较 5 和 7（1 次），选择 5；剩余元素 7 和 9 直接加入。② 合并(3, 5, 7, 9)和(6)：比较 3 和 6（1 次），选择 3；比较 5 和 6（1 次），选择 5；比较 7 和 6（1 次），选择 6；剩余元素 7 和 9 直接加入。总共比较 5 次。

例 8-5 C

12. C

基数排序有 MSD 和 LSD 两种，基数排序是稳定的。对于选项 A，不符合 LSD 和 MSD；对于选项 B，符合 MSD，但关键字 42、46 的相对位置发生了变化；对于选项 D，不符合 LSD 和 MSD。

例 8-6 3 段；最大 n，最小 m

05. 【解答】

1）当文件中的 n 个记录升序排列时，只生成一个归并段，长度达到最大，为 n。若初始工作区内的 m 个元素都大于输入文件中剩下的所有记录，则第一个归并段的长度就为 m，此时为第一个归并段长度最小的情况。排序过程如下表所示。

输出文件 FO	工作区 WA	输入文件 FI
—	—	51, 94, 37, 92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
—	51, 94, 37, 92	14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37	51, 94, 14, 92	63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51	94, 14, 92, 63	15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63	15, 94, 14, 92	99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92	15, 94, 14, 99	48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92, 94	15, 14, 99, 48	56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92, 94, 99	15, 14, 56, 48	23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92, 94, 99#	15, 14, 56, 48	23, 60, 31, 17, 43, 8, 90, 166, 100
14	15, 23, 56, 48	60, 31, 17, 43, 8, 90, 166, 100
14, 15	60, 23, 56, 48	31, 17, 43, 8, 90, 166, 100
14, 15, 23	60, 31, 56, 48	17, 43, 8, 90, 166, 100
14, 15, 23, 31	60, 17, 56, 48	43, 8, 90, 166, 100
14, 15, 23, 31, 48	60, 17, 56, 43	8, 90, 166, 100
14, 15, 23, 31, 48, 56	60, 17, 8, 43	90, 166, 100
14, 15, 23, 31, 48, 56, 60	90, 17, 8, 43	166, 100
14, 15, 23, 31, 48, 56, 60, 90	166, 17, 8, 43	100
14, 15, 23, 31, 48, 56, 60, 90, 166	100, 17, 8, 43	—
14, 15, 23, 31, 48, 56, 60, 90, 166#	100, 17, 8, 43	—
8	100, 17, 43	—
8, 17	100, 43	—
8, 17, 43	100	—
8, 17, 43, 100	—	—
8, 17, 43, 100#	—	—

生成三个初始归并段，分别是 37, 51, 63, 92, 94, 99；14, 15, 23, 31, 48, 56, 60, 90, 166；8, 17, 43, 100。

2）最大值为 n，最小值为 m。

例 8-7　825 次；反复合并当前两个最短表

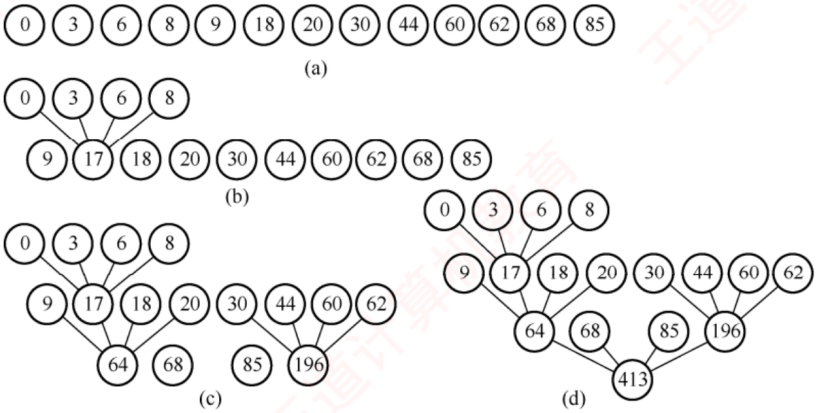
依次合并 `10+35=45`、`40+45=85`、`50+60=110`、`85+110=195`、`195+200=395`。每次最坏比较数为新段长减 1，合计 $44+84+109+194+394=825$。合并长度和为 WPL=830，减 5 次合并亦得 825。对 N 表反复取最短两表，等价于以表长为叶权构造哈夫曼树，使重复参与层数的加权和最小。

例 8-8　WPL=690

04.　【解答】

设初始归并段个数 $n=12$，外归并路数 $k=4$，计算 $(n-1)\%(k-1)=11\%3=2\neq 0$，说明不能做完全的 4 路归并，因为多出了两个初始归并段，必须添加 $k-2-1=1$ 个长度为 0 的空归并段，才能构成严格的 4 路归并树，即每次归并都有 k 个归并段参加归并。

此时，归并树的内结点应有$(n-1+1)/(k-1)=12/3=4$ 个，如下图所示。



$$\text{WPL}=(3+6+8)\times 3+(9+18+20+30+44+60+62)\times 2+(68+85)\times 1=51+486+153=690。$$

例 8-9　随机划分选择第 10 小，平均时间 $O(n)$、迭代辅助空间 $O(1)$

用随机枢轴划分，只继续处理包含第 10 小这一秩的子区间，丢弃无需继续划分的一侧。确定分界后保留 10 个最小数，无须内部有序；相等值跨边界时按尚需个数取足。随机选择的期望工作量为 $O(n)$，在原数组迭代执行辅助空间 $O(1)$。也可用大小 10 的大根堆扫描，只有新值小于堆顶才替换、下滤，时间 $O(n\log 10)$、辅助空间 $O(10)$，适合流式输入。

计算机组成原理 2

计算机系统与程序执行的层次

例 1-1 A

11. A

硬件能直接执行的只能是机器语言（二进制编码），汇编语言是增强机器语言的可读性和记忆性的语言，经过汇编后才能被执行。

例 1-2 A，平均 CPI=2.8，CPU 时间 28 μs。

19. A

CPI 指平均每条指令的执行需要多少个时钟周期。80%的指令执行平均需要 1 个时钟周期，20%的指令执行平均需要 10 个时钟周期，因此 CPI = 80%×1 + 20%×10 = 2.8。计算机主频为 1GHz，程序 P 共执行 10000 条指令，平均每条指令需要 2.8 个时钟周期，因此，CPU 执行时间 = $(10000\times2.8)\div10^9 = 2.8\times10^{-5}$ s = 28μs。

例 1-3 D，70 s。

14. D

程序 A 的运行时间为 100s，减去 CPU 时间 90s，剩余 10s 为 I/O 时间。CPU 提速 50%后运行基准程序 A 所耗费的时间是 $T = 90\div1.5 + 10 = 70$s。

误 区

CPU 速度提高 50%，而误认为 CPU 时间减少一半，从而误选选项 A。

数据表示、位级运算与浮点数

例 2-1 B，211.4。

01. B

十进制数转换为八进制数，整数部分采用除基取余法：将整数除以 8，所得余数即为转换后的八进制数个位上的数码，再将商除以 8，余数为八进制数十位上的数码，如此反复进行，直到商是 0 为止。小数部分采用乘基取整法：将小数乘以 8，所得积的整数部分即为八进制数十分位上的数码，再将此积的小数部分乘以 8，得到百分位上的数码，如此反复直到积是 1.0 为止。经转换得到的八进制数为 211.40。

例 2-2 B，0000 FFFAH。

27．B

将一个 16 位 unsigned short 型数转换为 32 位 unsigned int 型数时，因为都是无符号数，新表示形式的高位用 0 填充。16 位无符号整数所能表示的最大值为 65535，其十六进制表示为 FFFFH，因此 x 的十六进制表示为 FFFFH－5H＝FFFAH，所以 y 的十六进制表示为 0000 FFFAH。

排除法：先直接排除 C、D，然后分析余下选项的特征。A、B 的值相差几乎近 1 倍，因此可以算出 0001 0000H（接近 B 且好算的数）的值后，再推断出答案。

例 2-3 D。

34．D

在 32 位系统中，short 通常为 16 位有符号整数。−32767 的 16 位补码为 1000 0000 0000 0001。将其赋值给 32 位 unsigned int 时，先按**符号扩展**转换为 32 位有符号整数 1111 1111 1111 1111 1000 0000 0000 0001，再以无符号方式解释，其值为 $2^{32}-(2^{15}-1)=2^{32}-2^{15}+1$。

例 2-4 B。

18．B

不管是补码减法，还是无符号数减法，都是用被减数加上减数的负数的补码来实现的。根据求补公式，减数 y 的负数的补码$[-y]_{补}=\overline{Y}+1$，因此，在加法器的 Y' 输入端用一个反向器实现，并用控制端 Sub 控制多路选择器是否将 y 的各位取反后，输入 Y' 端，同时将 Sub 作为低位进位送到加法器。当 Sub 为 1 时，做减法，Sub＝1 控制将 $\overline{Y}$ 输入加法器 Y' 端，即实现“各位取反”功能；同时将 Sub ＝ 1 作为低位进位送到加法器，实现“末位加 1”功能。69 的二进制数为 0100 0101；38 的二进制数为 0010 0110，各位取反得 1101 1001。做减法时，低位进位为 Sub，即为 1。

注　意

若仅记忆补码加减运算的过程，而未掌握加法电路的原理，则本题易误选 D。

例 2-5 C。

29．C

利用补码转换为原码的规则：负数的符号位不变数值位取反加 1；正数补码等于原码。两个机器数对应的原码是$[x]_{原}$ = 80000021H，对应的数值是−33，$[y]_{原}=[y]_{补}$ = 00000041H＝65。排除选项 A、D。$x-y$ 直接利用补码减法准则，$[x]_{补}-[y]_{补}=[x]_{补}+[-y]_{补}$，$-y$ 的补码是连同符号位取反加 1，最终减法变成加法，得出结果为 FFFFFF9EH。

例 2-6 B，OF=0、CF=1。

32．B

ALU 生成标志位时只负责计算，而不管运算对象是有符号数还是无符号数，CF = 1 表示当作无符号数运算时溢出，OF = 1 表示当作有符号数运算时溢出。当作有符号数时，$x = 10$，$y = -20$，$x - y = 30$，未超过 32 位有符号数范围，不溢出，OF = 0。当作无符号数时，$x' = 10$，$y' = 2^{32} - 20$（符号位读作数值位），$x' - y' = 30 - 2^{32}$，为负，超过 32 位无符号数范围，溢出，CF = 1。

例 2-7 A，BEE0 0000H。

40．A

IEEE 754 单精度浮点数格式中依次为符号 1 位、阶码 8 位（偏置值 127）、尾数 23 位（隐藏 1 位）。$-0.4375 = -1.75 \times 2^{-2}$，保证小数点前是 1。根据单精度浮点数格式，符号为 1；阶码为移码表示，$-2 + 127 = 125$，写成 8 位二进制数为 01111101；尾数隐藏小数点前的 1，剩下的 0.75 写成二进制数为 0.11，所以尾数部分是 1100…0。该浮点数的二进制格式为 1011 1110 1110 0000 0000 0000 0000 0000，对应的十六进制格式为 BEE0 0000H。

例 2-8 A，x<y 且符号相同。

32．A

(f1)和(f2)对应的二进制分别是$(110011001001\ldots)_2$和$(101100001100\ldots)_2$，根据 IEEE 754 浮点数标准，可知(f1)的符号为 1，阶码为 10011001，尾数为 1.001，而(f2)的符号为 1，阶码为 01100001，尾数为 1.1，可知两数均为负数，符号相同，B、D 排除；(f1)的绝对值为 1.001×2^{26}，(f2)的绝对值为 1.1×2^{-30}，(f1)的绝对值比(f2)的绝对值大，而符号为负，真值大小相反，即(f1)的真值比(f2)的真值小，即 x < y。

例 2-9 D，I、II、III、IV。

33．D

对阶是较小的阶码向较大的阶码对齐，所以对阶后的阶码是当前那个较大的阶码而不会导致阶码溢出，说法 I 正确。右规和尾数舍入过程，阶码加 1 而可能上溢，说法 II 正确，同理说法 III 也正确。尾数溢出时可能仅产生误差，结果不一定溢出，说法 IV 正确。

例 2-10 （1）机器模型下循环条件恒真；改 int 后不进入循环。（2）相等；00FF FFFFH、4B7F FFFFH。（3）浮点舍入为 2^25。（4）低 32 位全 1；最大 n=30。（5）正无穷；不溢出最大 n=126，精确最大 n=23。

本题关于 f1 溢出后的返回值、n=0 的持续循环按题设 32 位补码机器模型推演；C 有符号溢出属未定义行为。

（1）unsigned 的 n-1 在 n=0 时为 $2^{32} - 1$，任意 unsigned i 均满足条件，i 递增可绕回 0。若 i、n 改 int，首次条件为 $0 \leq -1$，循环不执行。

（2）$f(23) = 2^{24} - 1 = 16777215$，int 和 24 位精度 binary32 均精确。int 机器数 00FF FFFFH；浮点 $1.\underbrace{11\cdots1}_{23位} \times 2^{23}$，阶码 150，机器数 4B7F FFFFH。

（3）$f(24) = 2^{25} - 1 = 33554431$ 需 25 位精度；binary32 相邻候选为 33554430、33554432，精确和居中，最近偶数取 33554432，即 4C00 0000H。

（4）$2^{32} - 1$ 超过 int 上界，机器模型保留低 32 位全 1，补码为 −1。精确须 $2^{n+1} - 1 \leq 2^{31} - 1$，故 n≤30。

（5）7F80 0000H 表示正无穷。n=126 的结果舍入为有限 2^{127}；n=127 越过有限上界并舍入为无穷。不溢出上界 126；完全精确需 n+1 个连续 1 全部容纳在 24 位中，上界 23。

例 2-11 D，0x11、0xC00E。

30．D

尽管 record 大小为 7B（成员 a 有 4B，成员 b 有 1B，成员 c 有 2B），因为数据按边界对齐方式存储，所以 record 共占用 8B。record.a 的十六进制表示为 0x00000111，因为采用小端方式存放数据，所以地址 0xC008 中的内容应为低字节 0x11；record.b 只占 1B，后面的 1B 留空；record.c 占 2B，因此其地址为 0xC00E。各字节的存储分配如下表所示。

地址	**0xC008**	**0xC009**	**0xC00A**	**0xC00B**
内容	record.a (0x11)	record.a (0x01)	record.a (0x00)	record.a (0x00)
地址	**0xC00C**	**0xC00D**	**0xC00E**	**0xC00F**
内容	record.b	–	record.c	record.c

存储层次、地址与 Cache

例 3-1 A，19 根。

32．A

4M×8 位的芯片数据线应为 8 根，地址线应为 $\log_2 4M = 22$ 根，而 DRAM 采用地址复用技术，地址线是原来的 1/2，且地址信号分行、列两次传送。地址线数为 22/2 = 11 根，所以地址引脚与数据引脚的总数为 11 + 8 = 19 根，选择选项 A。此题需要注意 DRAM 采用的是传两次地址的策略，所以地址线为正常的一半，这是很多考生容易忽略的地方。

例 3-2 D，26 位。

12．D

主存按字节编址，地址空间大小为 64MB，MAR 的寻址范围为 $64M = 2^{26}$，因此是 26 位。实际的主存容量 32MB 不能代表 MAR 的位数，考虑到存储器扩展的需要，MAR 应保证能访问到整个主存地址空间，反过来，MAR 的位数决定了主存地址空间的大小。

例 3-3 D，8000 和 8004。

34．D

每个访存地址对应的存储模块序号（0, 1, 2, 3）如下所示：

访存地址	8005	8006	8007	8008	8001	8002	8003	8004	8000
模块序号	1	2	3	0	1	2	3	0	0

其中，模块序号 = 访存地址%存储器交叉模块数。

判断可能发生访存冲突的规则如下：给定的访存地址在相邻的四次访问中出现在同一个存储模块内。据此，根据上表可知 8004 和 8000 对应的模块号都为 0，即表明这两次的访问出现在同一模块内且在相邻的访问请求中，满足发生冲突的条件。

例 3-4 D，ROM 2 片、RAM 30 片。

10．D

首先确定 ROM 的个数，ROM 区为 4KB，选用 2K×8 位的 ROM 芯片，需要(4K×8)÷(2K×8) = 2 片，采用字扩展方式；RAM 区为 60KB，选用 4K×4 位的 RAM 芯片，需要(60K×8)÷(4K×4) = 30 片，采用字和位同时扩展的方式。

例 3-5 B，约 12.2 ms。

11．B

存取时间 = 寻道时间 + 旋转等待时间 + 传输时间。存取一个扇区的平均旋转等待时间为旋转半周的时间，即(60/7200)/2 = 4.17ms，传输时间为(60/7200)/1000 = 0.01ms，因此访问一个扇区的平均存取时间为 4.17 + 0.01 + 8 = 12.18ms，保留一位小数则为 12.2ms。

例 3-6 C，约 12.5%。

32．C

分析语句“a[k] = a[k] + 32”：首先读取 a[k]需要访问一次 a[k]，之后将结果赋值给 a[k]需要访问一次，共访问两次。第一次访问 a[k]未命中，并将该字所在的主存块调入 Cache 对应的块中，对该主存块中的 4 个整数的两次访问中，只在访问第一次的第一个元素时发生缺失，其他的 7 次访问中全部命中，因此该程序段执行过程中访问数组 a 的 Cache 缺失率约为 12.5%。

例 3-7 A，8 个 20 位比较器。

35．A

Cache 采用组相联映射，主存地址结构应分为标记、组号、块内地址三部分。主存块大小 = Cache 块大小 = 64B = 2^6B，因此块内地址占 6 位。Cache 数据区容量为 32KB，每个 Cache 块大小为 64B，则 Cache 总块数 = 32KB/64B = 2^9，因为采用 8 路组相联映射，即每 8 个 Cache 块为一个分组，所以共被分为 $2^9/8 = 2^6$ 组，因此，组号占 6 位。除了块内地址和组号，剩余的位为标记位，占 32 − 6 − 6 = 20 位。地址结构如下所示。

标记	组号	块内地址
20 位	6 位	6 位

Cache 采用 8 路组相联映射，因此在访问一个物理地址时，要先根据组号定位到某一分组，然后用物理地址的高 20 位（标记）与分组中 8 个 Cache 行的标记做并行比较（用 8 个 20 位“比较器”实现），若某个 Cache 行的标记与物理地址的高 20 位完全一致，则选中该 Cache 行。综上所述，在组相联映射的 Cache 中，“比较器”用于并行地比较分组中所有 Cache 行的标记位与要访问物理地址的标记位，因此比较器的个数就是分组中的 Cache 行数 8，比较器的位数就是标记位数 20。

例 3-8 C，148 K 位。

31．C

直接映射的地址结构为

主存字块标记	Cache 字块标记	字块内地址

按字节编址，块大小为 4×32 位 ＝16B＝2^4B，则“字块内地址”占 4 位；“能存放 4K 字数据的 Cache”即 Cache 的存储容量为 4K 字（注意单位），则 Cache 共有 1K ＝ 2^{10} 个 Cache 行，Cache 字块标记占 10 位；主存字块标记占 32－10－4＝18 位。

Cache 总容量包括：存储容量和标记阵列容量（有效位、标记位、脏位和替换算法控制位）。标记阵列中的有效位和标记位一定存在，而脏位和替换算法控制位的取舍需要看题意，题目中明确说明了采用回写法，则一定包含一致性维护位，而关于替换算法的词眼题目中未提及，所以不予考虑。因此，每个 Cache 行标记项包含 18＋1＋1＝20 位，标记阵列容量为 2^{10}×20 位 ＝20K 位，存储容量为 4K×32 位 ＝128K 位，总容量为 128K＋20K＝148K 位。

例 3-9 C，01 8840H。

17．C

页大小为 4KB＝2^{12}B，按字节编址，因此页内地址为 12 位。虚拟地址空间大小为 4GB＝2^{32}B，因此虚拟地址共 32 位，其中低 12 位为页内地址，高 20 位为虚页号。题中给出的虚拟地址为 0008 2840H，虚页号为高 20 位即 00082H（页内地址为低 12 位即 840H），82H 对应的十进制数为 130（注意题中页表的虚页号部分末尾未写 H，所以是十进制数，因此查找时要先将虚页号转换为十进制数），查页表命中，并且存在位为 1，对应页框号为 018H。将查找到的页框号 018H 和页内地址 840H 拼接，得到主存地址为 01 8840H。

例 3-10 C，19 位。

19．C

按字节编址，页大小为 2^{10}B，因此页内地址占 10 位；TLB 有 32 个表项，采用 4 路组相联映射，被分为 2^3＝8 组， 因此 TLB 组号占 3 位；于是，标记字段的位数至少是 32－3－10＝19。

例 3-11 （1）2 页、2 次缺页，0042 2000H 与 0042 3000H。（2）以数组元素计，无时间局部性，有空间局部性。（3）块内 A4～A0，组号 A10～A5；a[1][0]=0042 2100H，组号 8。（4）192 块；写分配下两种循环均 87.5%。

命中率按写分配、无其他数据冲突的模型计算。原题未显式给出写分配，采用非写分配时须重算。

（1）数组 $24 \times 64 \times 4 = 6144$ B，起址页对齐，覆盖 2 页。原循环第一次访问第一页 a[0][0]、第二页 a[16][0] 时分别缺页，故障地址为 0042 2000H、0042 3000H；页面不置换，共 2 次。

（2）每元素只写一次，以元素为对象无时间局部性；按行顺序访问相邻元素具有空间局部性。块内复用体现空间局部性。

（3）块长 32 B，偏移 5 位；$8192/(32 \times 4) = 64$ 组，组号 6 位，即 A10～A5。这些位均在 12 位页内偏移内，虚实转换保持不变。每行 256 B，a[1][0] 为 0042 2100H，组号 $(2100\text{H} \gg 5) \bmod 64 = 8$。

（4）块数 6144/32=192，元素 1536。按行每块首次写缺失，随后 7 次命中，命中率 $1 - 192/1536 = 87.5\%$。按列时，同一列相邻元素跨 8 块，24 行只映射 8 组，每组 3 块，可驻留于 4 路；后续同块各列继续命中。不同 8 列批次使用不同组，仍仅 192 次首次缺失，命中率 87.5%。

例 3-12 （1）VPN=A29～A12，偏移 =A11～A0。（2）标记 =A29～A15，组号 =A14～A12。（3）淘汰 VPN=4。（4）每表项增加 2 位。

07. 【解答】

注意：对于本题的 TLB，需要采用处理 Cache 的方式求解。

1）按字节编址，页面大小为 4 KB = 2^{12}B，页内地址为 12 位。虚拟地址中高 30 – 12 = 18 位表示虚页号，虚拟地址中低 12 位表示页内地址。

2）TLB 采用 2 路组相联映射方式，共 8 = 2^3 组，用 3 位来标记组号。虚拟地址（或虚页号）中高 18 – 3 = 15 位为 TLB 标记，虚拟地址中随后 3 位（或虚页号中低 3 位）为 TLB 组号。

3）虚页号 4 对应的 TLB 表项被替换。因为虚页号与 TLB 组号的映射关系为 TLB 组号 = 虚页号 mod TLB 组数 = 虚页号 mod 8，因此，虚页号 10, 12, 16, 7, 26, 4, 12, 20 映射到的 TLB 组号依次为 2, 4, 0, 7, 2, 4, 4, 4。TLB 采用 2 路组相联映射方式，从上述映射到的 TLB 组号序列可以看出，只有映射到 4 号组的虚页号数量大于 2，相应虚页号依次是 12, 4, 12 和 20。根据 LRU 算法，当访问第 20 页时，虚页号 4 对应的 TLB 表项被替换出来。

4）虚拟地址位数增加到 32 位时，虚页号增加了 32 – 30 = 2 位，使得每个 TLB 表项中的标记字段增加 2 位，因此，每个 TLB 表项的位数增加 2 位。

例 3-13 （1）虚址 6/2/6 位，物理址 6/4/2 位。（2）03D4H 读得 36H；00F1H 从主存取数，字节值未知；0229H 缺页。

（1）64 B 页占 6 位偏移，VPN 为 8 位；TLB16 项、4 路共 4 组，组号 2 位、标记 6 位。Cache 块 4 B 占 2 位偏移，16 行占 4 位行号，物理址剩 6 位标记。

（2）每次均从题给初态开始，查表项时同时检查有效位和标记。

虚拟地址	VPN ／偏移	TLB 组／标记	转换与数据路径
03D4H	0FH ／ 14H	3 ／ 03H	组 3 路 2 命中，页框 0DH；物理地址 354H；Cache 标记 0DH、行 5H、偏移 0，命中读 36H
00F1H	03H ／ 31H	3 ／ 00H	TLB 缺失，页表 P=1、页框 02H；物理地址 0B1H；Cache 标记 02H、行 CH、偏移 1，有效位 0，从主存 0B0H～0B3H 调块后取偏移 1，字节值未给出
0229H	08H ／ 29H	0 ／ 02H	TLB 缺失，页表 P=0，缺页；待处理并重试后继续

第一地址计算为 `0DH` × `40H` + `14H` = `354H`；第二为 `02H` × `40H` + `31H` = `0B1H`。第三次在取得驻留页框前进入缺页处理，不继续当前普通数据 Cache 查找。

指令系统与 C 程序的机器级表示

例 4-1 D，128 条。

15．D

地址码为 6 位，一条二地址指令会占用 2^6 条一地址指令的空间，一条一地址指令会占用 2^6 条零地址指令的空间。若全都是零地址指令，则最多有 2^{16} 条，减去一地址指令和二地址指令所占用的零地址指令空间，即 $2^{16}-254\times2^6-12\times2^6\times2^6=(2^{10}-254-12\times2^6)\times2^6=2\times2^6=128$。

【另解】二地址指令有 12 条，则剩余 16 − 12 = 4 种操作码给一地址指令，一地址指令有 254 条，剩余 4×64 − 254 = 2 种操作码给 0 地址指令，所以 0 地址一共有 $2\times2^6=128$ 条。

例 4-2 C，2008H。

23．C

相对寻址 EA = (PC) + A，首先计算取指令后的 PC 值。转移指令由 2 字节组成，每取一字节 PC 加 1，取指令后的 PC 值为 2002H，因此 EA = (PC) + A = 2002H + 06H = 2008H。本题易误选选项 A 或 B，选项 A 未考虑 PC 值的自动更新，选项 B 虽然考虑了 PC 值的自动更新，但未注意到该转移指令是一条 2 字节指令，PC 值应是“+2”而不是“+1”。

例 4-3 C。

06．C

RISC 指令集具有格式规整、数量少、寻址简单、采用 LOAD/STOR 访存、通用寄存器数量多及硬连线控制器等特点，指令执行周期均匀，非常适合流水线实现。相反，CISC 因指令复杂、长度可变而难以采用流水线。因此，选项 C 错误，其余选项均符合 RISC 特征。

例 4-4 A。

14．A

将源程序转换为可执行目标文件的过程分为预处理、编译、汇编、链接四个阶段。

例 4-5 （1）10 次，第 16 行；（2）第 12 行条件转移，第 16、20、30 行无条件改变后继；（3）0040102AH，位移 –42，小端；（4）32 位溢出，改用 long long；（5）高 32 位不等于低 32 位的符号扩展时 OF=1，增加溢出自陷指令。

（1）调用参数依次 10、9、…、1，共 10 次（含最外层）。第 16 行 call 递归调用，执行 9 次。

（2）第 12 行 jle 为条件转移；第 16 行 call、第 20 行 jmp、第 30 行 ret 执行时均改变顺序后继地址。

（3）call 占 5 B，后继地址为 $00401025\text{H}+5=0040102A\text{H}$。相对位移 $00401000\text{H}-0040102A\text{H}=-2A\text{H}=-42$，32 位补码为 FFFF FFD6H。内存低地址先存 D6，故为小端。

（4）$13!>2^{31}-1$，超出 32 位 int 范围。题给机器结果为截低 32 位：$6227020800-2^{32}=1932053504$。标准 C 有符号溢出规则不保证这种回绕。将返回类型改为至少 64 位的 long long，使递归返回值与乘法均按宽类型计算：

```
long long f1(int n) {
    if (n > 1) return n * f1(n - 1);
    return 1LL;
}
```

(5) 令精确 64 位积的高、低 32 位分别为 H、L。当 H 等于 L 符号位的 32 位扩展（L 非负则 H 全 0，L 负则 H 全 1），OF=0；其他情况 OF=1。编译器可在 imul 后加入检查 OF 的溢出自陷指令（32 位 x86 为 `into`），OF=1 时进入异常处理。

中央处理器

例 5-1 B。

18. B

组合逻辑元件（操作元件）不含存储信号的记忆单元，任何时刻产生的输出仅取决于当前的输入，加法器、算术逻辑单元（ALU）、译码器、多路选择器、三态门等都属于操作元件。时序逻辑元件（状态元件）包含存储信号的记忆单元，各类寄存器和存储器，如通用寄存器组、程序计数器、状态/移位/暂存/锁存寄存器等，都属于状态元件。

例 5-2 （1）SF=F15，OF 见解析；（2）Y 保存首操作数，Z 暂存结果；（3）16 个，来自 IR，rd 接写地址译码器；（4）PCout/MARin→Read→MDRout/IRin，至少 7 拍；（5）CU 产生，IR 和 FR 提供输入。

（1）令 $a = A_{15}, b = B_{15}, f = F_{15}$，乘为逻辑与、加为逻辑或：

$$\mathrm{SF} = f, \quad \mathrm{OF}_{add} = ab\overline{f} + \overline{a}\,\overline{b}f, \quad \mathrm{OF}_{sub} = a\overline{b}\,\overline{f} + \overline{a}bf.$$

加法同号输入异号结果时溢出；减法异号输入、结果符号异于 A 时溢出。

（2）总线同拍仅提供一个值，Y 暂存首操作数，随后总线提供第二个。结果先存 Z，避免结果与第二输入同时驱动总线，再由 Z 输出。

（3）rs、rd 为 4 位，可选 $2^4 = 16$ 个寄存器，来自 IR 字段。rd 接写地址译码器，rs 控制读多路选择器。

（4）PCout、MARin（1 拍）→Read，主存读并送 MDR（5 拍）→MDRout、IRin（1 拍），至少 7 拍。主存读入 MDR 已计在 5 拍内；MDRin 为内部总线写 MDR，本次主存路径无需该信号。

（5）信号由 CU 产生，IR 的指令信息和 FR 的状态标志输出连到 CU 输入。

例 5-3 B。

25. B

该指令的两个源操作数分别采用寄存器、寄存器间接寻址方式，因此在取数阶段需要用到通用寄存器组（GPRs）和存储器（Memory）；在执行阶段，两个源操作数相加需要用到算术逻辑单元（ALU）。而指令译码器（ID）用于对操作码字段进行译码，向控制器提供特定的操作信号，在取数及执行阶段用不到。

例 5-4 C，15 位。

22．C

字段直接编码法将微命令字段分成若干小字段，互斥性微命令组合在同一字段中，相容性微命令分在不同字段中，每个字段还要留出一个状态，表示本字段不发出任何微命令。5 个互斥类，分别包含 7、3、12、5 和 6 个微命令，需要 3、2、4、3 和 3 位，共 15 位。

例 5-5 C，8 位。

23．C

计算机共有 32 条指令，各个指令对应的微程序平均为 4 条，则指令对应的微指令为 $32\times4=128$ 条，而公共微指令还有 2 条，整个系统中微指令的条数共为 $128+2=130$ 条，所以需要 $\lceil\log_2 130\rceil=8$ 位才能寻址到 130 条微指令。

例 5-6 B。

13．B

系统调用属于自陷，“断点”为自陷指令的下一条指令地址。DMA 传送结束后，DMA 控制器需要向 CPU 发送中断请求，属于外中断，外中断的“断点”为下一条指令地址。打印机缺纸同样属于外中断。页缺失属于内部异常中的故障，“断点”为发生故障的指令地址，执行完缺页异常处理程序之后必须返回发生故障的指令重新执行。

例 5-7 A，90 ns。

共同节拍至少覆盖最慢功能段，$\tau \geq \max(90,80,70,60)=90$ ns，选 A；题目已忽略段寄存器开销。

例 5-8 D，1 条。

无停顿时 I2 在第 5 拍 MEM 末才有 r3，I3 原本第 5 拍 EX 就需使用，需把 I3 的 EX 推迟到第 6 拍。I1 的 r1 已有可用结果，转发可满足。因此插 1 条 NOP，选 D。

例 5-9 C，I3、I4。

30．C

I2 和 I1 之间存在数据冒险，I1 在 WB 段才将新值写回寄存器 R[s2]，但 I2 的 EX 段就要读 R[s2]以计算访存的有效地址，I1 在 EX 段结束时就已生成 R[s2]的新值，被存放在 EX/MEM 流水段寄存器中，采用转发技术后，可直接从该寄存器中取出数据送到 ALU 的输入端，这样 I2 执行时 ALU 用的是 R[s2]的新值，解决了 I2 和 I1 之间的数据冒险。I3 和 I2 之间存在数据冒险，属于 load-use 数据冒险，用转发技术无法解决 I3 和 I2 的数据相关问题，原因在于 I2 的功能是从内存中取数，只有在 MEM 段结束时才能从主存中得到 R[s3]的新值，但 I3 的 EX 段就要用到 R[s3]，因此无法用转发技术解决，I3 仍需阻塞一个时钟周期，等到 I2 的 MEM 段结束后，从 I2 的 MEM/WB 流水段寄存器中取到 R[s3]的新值。I4 和 I3 之间存在控制冒险，beq 指令在 EX 段设置条件码，在 MEM 段控制是否将转移地址送到 PC，这之后才能开始根据 PC 内容取指令，因此 I4 需要进行硬件阻塞。综上所述，I3 和 I4 的执行需要被阻塞，指令执行过程如下。

I1	IF	ID	EX	MEM	WB							
I2		IF	ID	EX	MEM	WB						
I3			IF	ID	阻塞	EX	MEM	WB				
I4/I5				阻塞	阻塞	阻塞	阻塞	IF	ID	EX	MEM	WB

例 5-10 C。

12．C

MIMD 结构分为多计算机系统和多处理器系统。向量处理器是 SIMD 的变体，属于 SIMD 结构。硬件多线程技术在一个核中处理多个线程，可用于单核处理器，选项 C 错误。共享内存多处理器（SMP）具有共享的单一物理地址空间，所有核都可通过存取指令来访问同一片主存地址空间。

例 5-11 （1）FEFFH；（2）8 拍；（3）I3 等待 RAW 写回，I4 受前级冻结阻塞；（4）至少 17 拍。

统一约束：无转发，同寄存器不能同拍读写，消费者 ID 最早为生产者 WB 的下一拍；按题设计所有指令流出五段。

（1）−513 的 16 位补码 FDFFH，算术右移复制符号位，得到 FEFFH（−257）。负奇数算术右移按位结果对应向负无穷取整。

（2）$5+4-1=8$ 拍。

（3）I3 读 R1、R2，分别依赖 I1 的 WB5、I2 的 WB6，故 ID 须到第 7 拍，第 4～6 拍等待。IF/ID 保持使 I4 也不能取指。I3 的 WB10 后 I4 最早 ID11，因此流出为第 14 拍。I4 的 IF 阻塞由前级冻结传播产生。

（4）先装 x 以提前关键链：

```
I1 LOAD  R1, [x]
I2 LOAD  R2, [a]
I3 SHL   R1
I4 ADD   R1, R2
I5 STORE R2, [x]
```

指令	IF	ID 成功	EX	M	WB/流出
I1	1	2	3	4	5

指令	IF	ID 成功	EX	M	WB/流出
I2	2	3	4	5	6
I3	3	6	7	8	9
I4	6	10	11	12	13
I5	10	14	15	16	17

I3 在 ID 端等第 4～5 拍，I4 等第 7～9 拍，I5 等第 11～13 拍，相应后继取指冻结。x 的 LOAD→ 翻倍 → 相加 →STORE 链使这三次消费者 ID 分别至少第 6、10、14 拍，所列调度达到下界，共 17 拍。STORE 在 M16 已写主存，按五段流出计为 17 拍。

总线

例 6-1 B，20 MB/s。

16．B

总线带宽是指单位时间内总线上传输数据的位数，通常用每秒传送信息的字节数来衡量，单位为 B/s。由题意可知，在 1 个总线周期（=2 个时钟周期）内传输了 4 字节信息，时钟周期 = 1/10MHz = 0.1μs，因此总线带宽为 4B÷(2×0.1μs) = 4B÷(0.2×10^{-6}s) = 20MB/s。

例 6-2 10.664 GB/s；改给 333.25 MHz 且每拍 4 次时相同。

64 位为 8 B，1333 MT/s 已是每秒传送次数，$B = 1333 \times 10^6 \times 8 = 10.664$ GB/s。若给 333.25 MHz，则 $R = 4 \times 333.25 = 1333$ MT/s，结果相同。

例 6-3 D，32 ns。

20．D

每次传输需经过传输地址、准备数据和传输数据三个过程，分别需要 1ns（时钟频率为 1GHz，因此时钟周期为 1ns）、6ns 和 1ns，共 8ns。总线宽度为 64 位，所以每次传输的数据为 64 位，主存块大小为 32B，所以读取一个主存块需要传输 4 次，即 8ns×4 = 32ns。

例 6-4 B，6.72 GB/s。

24．B

总线带宽（最大数据传输率）是理想情况（所有总线周期都在传送数据）下，不需考虑每个总线事务的具体情况，而计算（平均）数据传输率才需考虑每个总线事务的具体情况。因此，题中“其总线事务支持……连续传送 8 次数据。”这句话属于干扰条件。根据定义，总线带宽 = 总线宽度×总线时钟频率×每个时钟周期传送数据的次数 = 64bit×420MHz×2 = 6.72GB/s。

输入/输出系统

例 7-1 D。

14．D

I/O 总线分为三类：数据线、控制线和地址线。数据缓冲寄存器和命令/状态寄存器的内容都是通过数据线来传送的；地址线用以传送与 CPU 交换数据的端口地址；而控制线用于给 I/O 端口发送读/写信号，仅用于对端口进行读/写控制。中断类型号用于指出中断向量的地址，CPU 响应某一外部中断后，就从数据总线上获取该中断源的中断类型号，然后据此计算对应中断向量在中断向量表（存放在内存中）的位置。因此说法 I、II 和 III 均正确。

例 7-2 C，0.20%。

36．C

每秒至少查询 200 次，每次查询至少 500 个时钟周期，则每秒最少占用 200×500 = 100000 个时钟周期，因此占 CPU 时间的百分比为 100000/50M = 0.20%。

例 7-3 A、B、C 屏蔽字依次 100、110、111；开始 A→B→C，完成 C→B→A。

位序 A、B、C，1 为屏蔽。A 允许 B、C 打断，屏蔽字 100；B 只允许 C，110；C 全屏蔽，111。初始均开放，硬件先选 A；A 开中断后在 B、C 中先选 B，B 开中断后进入 C。C 完成恢复 B，B 完成恢复 A；各层恢复自己的旧屏蔽字，最终按 C、B、A 完成。

例 7-4 B，25%。

39．B

每 400ns 发出一次中断请求，而响应和处理时间为 100ns，其中允许的延迟为干扰信息，因为在 50ns 内，无论怎么延迟，每 400ns 仍要花费 100ns 处理中断，所以该设备的 I/O 时间占整个 CPU 时间的百分比为 100ns/400ns = 25%。

例 7-5 C。

47．C

周期挪用法由 DMA 控制器挪用一个或几个主存周期来访问主存，传送完一个数据字后立即释放总线，是一种单字传送方式，每个字传送完后 CPU 可以访问主存，选项 C 错误。停止 CPU 访存法则是指在整个数据块的传送过程中，使 CPU 脱离总线，停止访问主存。

例 7-6 D。

44. D

每类设备都配置一个设备驱动程序，设备驱动程序向上层用户程序提供一组标准接口，负责实现对设备发出各种具体操作指令，用户程序不能直接和 DMA 打交道。DMA 的数据传送过程分为预处理、数据传送和后处理 3 个阶段。预处理阶段由 CPU 完成必要的准备工作，数据传送前由 DMA 控制器请求总线使用权；数据传送由 DMA 控制器直接控制总线完成；传送结束后，DMA 控制器向 CPU 发送中断请求，CPU 执行中断服务程序做 DMA 结束处理。

例 7-7 （1）最多 2 μs 查询一次，至少 4%；（2）不能，需 800% 的 CPU 预算；（3）最多 4%。

10. 【解答】

1）程序定时向缓存端口查询数据，由于缓存端口大小有限，必须在传输完端口大小的数据时访问端口，以防止部分数据未被及时读取而丢失。设备 A 准备 32 位数据所用的时间为 4B/2MB = 2μs，所以最多每隔 2μs 必须查询一次，每秒的查询次数至少是 1s/2μs = 5×10^5，每秒 CPU 用于设备 A 输入/输出的时间至少为 $5\times10^5\times10\times4=2\times10^7$ 个时钟周期，占整个 CPU 时间的百分比至少是 $2\times10^7\div500M = 4\%$。

2）中断响应和中断处理的时间为 400×(1/500M) = 0.8μs，这时只需判断设备 B 准备 32 位数据要多久，若准备数据的时间小于中断响应和中断处理的时间，则数据被刷新，造成丢失。经过计算，设备 B 准备 32 位数据所用的时间为 4B/40MB = 0.1μs，因此设备 B 不适合采用中断 I/O 方式。

3）在 DMA 方式中，只有预处理和后处理需要 CPU 处理。设备 B 每秒的 DMA 次数最多为 40MB/s÷1000B = 40000，CPU 用于设备 B 输入/输出的时间最多为 $40000\times500 = 2\times10^7$ 个时钟周期，占 CPU 总时间的百分比最多为 $(2\times10^7)/500M = 4\%$。

3 操作系统

计算机系统概述

例 1-1 D

20. D

操作系统的基本特点：并发、共享、虚拟、异步，其中最基本、一定要实现的是并发和共享。早期的多道批处理系统会将所有进程的数据全部调入主存，再让多道程序并发执行，即使不支持虚拟存储管理，也能实现多道程序并发。进程多并不意味着 CPU 的利用率高，进程数量越多，进程之间的资源竞争越激烈，甚至可能因为资源竞争而出现死锁现象，导致 CPU 的利用率低。

例 1-2 B

31. B

发生系统调用时，CPU 通过执行软中断指令将 CPU 的运行状态从用户态切换到内核态，这个过程与中断和异常的响应过程相同，由硬件负责保存断点和程序状态字，并将 CPU 模式改为内核态。然后，执行操作系统内核的系统调用入口程序，该内核程序负责保存通用寄存器的内容，再调用执行特定的系统调用服务例程。综上，说法 I、IV 由硬件完成，说法 II、III 由操作系统完成。

例 1-3 D

21. D

微内核架构将内核中最基本的功能保留在内核，只有微内核运行在内核态，其余模块都运行在用户态，一个模块中的错误只会使这个模块崩溃，而不会使整个系统崩溃，因此具有较高的可靠性和安全性。微内核的非核心功能运行在用户空间，可通过插件或模块的方式进行扩展，无须改动内核代码，因此具有较强的可扩展性。微内核需要频繁地在用户态和内核态之间进行切换，操作系统的执行开销偏大，从而影响系统性能。

例 1-4 A

20. A

在操作系统初始化的过程中需要创建中断向量表，以实现通电自检（POST），CPU 检测到中断信号后，根据中断号查询中断向量表，跳转到相应的中断处理程序，选项 A 正确。在硬盘逻辑格式化之前，需要先对硬盘进行分区，即创建硬盘分区表。分区完成后，对物理分区进行逻辑格式化（创建文件系统），为每个分区初始化一个特定的文件系统，并创建文件系统的根目录。若某个分区采用 UNIX 文件系统，则还要在该分区中建立文件系统的索引节点表。

进程与线程

例 2-1 C

71. C

在等待键盘输入的操作中，当前线程处于阻塞态，键盘输入完成后，再调出相应的中断服务程序进行处理，由中断服务程序负责唤醒当前线程，选项 A 错误。当线程检测到缺页异常时，会调用缺页异常处理程序从外存调入缺失的页面，线程状态从运行态转换为阻塞态，选项 B 错误。当线程的时间片用完后，主动放弃 CPU，此时若线程还未执行完，就进入就绪队列等待下次调度，此时线程状态从运行态转换为就绪态，选项 C 正确。线程执行 wait()后，若成功获取资源，则线程状态不变，若未能获取资源，则线程进入阻塞态，选项 D 错误。

例 2-2 D

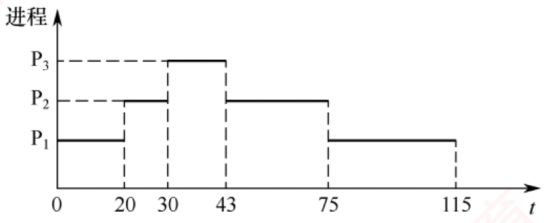
45. D

注意，系统是在 $t=2$ 时开始作业调度的，此时 J_4 还没有到达。FCFS 调度算法的特点是作业来得越早，优先级就越高，因此选择 J_1。SJF 调度算法的特点是作业运行时间越短，优先级就越高，因此选择 J_3。

例 2-3 B；61 ms

53. B

采用抢占式优先级调度算法，三个作业的执行顺序如右图所示。



周转时间 = 完成时间 – 到达时间。P_1 的周转时间 $=115-0=115$ms；P_2 的周转时间 $=75-20=55$ms；P_3 的周转时间 $=43-30=13$ms，所以平均周转时间 $=(115+55+13)/3=61$ms。

例 2-4 C；6 次

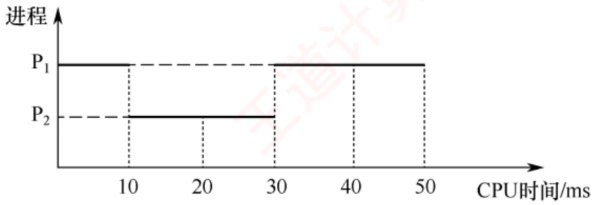
52. C

需要注意的是，在 0 时刻，P_0 获得 CPU 也是一次进程调度，所以 0 时刻调度进程 P_0 获得 CPU；10ms 时 P_2 进入就绪队列，调度 P_2 抢占获得 CPU；15ms 时 P_3 进入就绪队列，调度 P_3 抢占获得 CPU；25ms 时 P_3 执行完毕，调度 P_2 获得 CPU；40ms 时 P_2 执行完毕，调度 P_0 获得 CPU；130ms 时 P_0 执行完毕，调度 P_1 获得 CPU；190ms 时 P_1 执行完毕，结束；总共调度 6 次。

例 2-5 C；15 ms

48. C

进程 P_1 和 P_2 依次创建后进入队列 Q_1，根据时间片调度算法的规则，进程 P_1 和 P_2 将依次被分配 10ms 的 CPU 时间，两个进程分别执行完一个时间片后都会被转入队列 Q_2，就绪队列 Q_2 采用短进程优先调度算法，此时 P_1 还需要 20ms 的 CPU 时间，P_2 还需要 10ms 的 CPU 时间，所以 P_2 会被优先调度执行，10ms 后进程 P_2 执行完成，之后 P_1 再调度执行，再过 20ms 后 P_1 也执行完成。运行图表述如右。



进程 P_1、P_2 的等待时间分别为图中的虚横线部分，平均等待时间= (P_1 的等待时间 + P_2 的等待时间)/2 = (20 + 10)/2 = 15，因此答案选 C。

例 2-6 SRTF：13；RR：19.25

06.【解答】

1）按照可抢先式短进程优先调度算法，进程运行时间见下表。

进 程 名	到达就绪队列时刻	预计执行时间	执行时间段	周转时间
P_1	0	8	0～1；10～17	17
P_2	1	4	1～5	4
P_3	2	9	17～26	24
P_4	3	5	5～10	7

- 时刻 0，进程 P_1 到达并占用处理器运行。
- 时刻 1，进程 P_2 到达，因其预计运行时间短，因此抢夺处理器进入运行，P_1 等待。
- 时刻 2，进程 P_3 到达，因其预计运行时间长于正在运行的进程，进入就绪队列等待。
- 时刻 3，进程 P_4 到达，因其预计运行时间长于正在运行的进程，进入就绪队列等待。
- 时刻 5，进程 P_2 运行结束，调度器在就绪队列中选择短进程，P_4 符合要求，进入运行，进程 P_1 和进程 P_3 则还在就绪队列等待。
- 时刻 10，进程 P_4 运行结束，调度器在就绪队列中选择短进程，P_1 符合要求，再次进入运行，而进程 P_3 则还在就绪队列等待。
- 时刻 17，进程 P_1 运行结束，只剩下进程 P3，调度其运行。
- 时刻 26，进程 P_3 运行结束。

平均周转时间 $=[(17-0)+(5-1)+(26-2)+(10-3)]/4=13$。

2）时间片轮转调度算法按就绪队列的 FCFS 进行轮转，在时刻 2，P_1 被挂到就绪队列队尾，队列顺序为 P_2, P_3, P_1，此时 P_4 还未到达。按时间片轮转调度算法的进程时间分配见下表。

进 程 名	到达就绪队列时刻	预计执行时间	执行时间段	周转时间
P_1	0	8	0～2；6～8；14～16；20～22	22
P_2	1	4	2～4；10～12	11
P_3	2	9	4～6；12～14；18～20；23～25；25～26	24
P_4	3	5	8～10；16～18；22～23	20

平均周转时间 $=[(22-0)+(12-1)+(26-2)+(23-3)]/4=19.25$。

例 2-7 $priority = nice + k_1 cpuTime - k_2 waitTime$，$k_1, k_2 > 0$

（1）只比较 nice 时，较小 nice 的进程持续就绪可使较大 nice 的进程长期落选。

（2）取 $priority = nice + k_1 cpuTime - k_2 waitTime$，$k_1, k_2 > 0$。运行使优先数增大，等待使优先数减小；waitTime 实现老化。按题设在运行时清 waitTime、就绪时清 cpuTime。无饥饿保证还需有限进程或有界 nice、持续重算、适当抢占或有限运行段及公平同级处理等前提。

例 2-8 C

11. C

并发进程因共享资源而产生相互制约，可分为两类：① 互斥关系，指进程竞争使用独占资源（互斥资源）所形成的制约；② 同步关系，指进程为协同工作而需交换信息、相互等待所形成的制约。本题中，两个进程之间的制约属于同步关系，进程 B 必须等待进程 A 将数据写入缓冲区后，才能从中读取数据。同时，缓冲区是共享资源，其访问必须互斥，因此二者也存在互斥关系。

例 2-9 D

18. D

与互斥信号量初值一般置 1 不同，用 P, V 操作实现进程同步时，信号量的初值应根据具体情况来确定。若期望的消息尚未产生，则对应的初值应设为 0；若期望的消息已存在，则信号量的初值应设为一个非 0 的正整数。

例 2-10 两个信号量，初值均 0

T1 按 A、E、F 执行，T2 按 B、C、D 执行；本线程顺序已保证 B→C、C→D、E→F。跨线程只需 A→C 和 C→E。

```
semaphore ac=0, ce=0;
T1: A; V(ac); P(ce); E; F;
T2: B; P(ac); C; V(ce); D;
```

C 前已完成本线程的 B 并收到 A 通知；E 前收到 C 通知。V(ce) 放在 C 后、D 前，保留 D 与 E/F 的并发。

例 2-11 （1）是；（2）done=0；（3）mutex=1

（1）两个 C1 可同时发现空缓冲并互相覆盖，检查和写入须作为临界区保护。

（2）初始空且各执行一次，只需写完通知：

```
semaphore done=0;
P1: C1; V(done);
P2: P(done); C2;
```

该先后关系已排除两次操作重叠。

（3）初始非空且两次修改均不清空，使用互斥量：

```
semaphore mutex=1;
P1: P(mutex); C3; V(mutex);
P2: P(mutex); C3; V(mutex);
```

每问按各自初态和活动范围独立分析。

例 2-12　empty=N，odd=even=0，mutex=1；按类别发布

原题未明确初态，沿参考解取初始为空；有初存 a 个奇数、b 个偶数时改为 empty=N-a-b、odd=a、even=b。所有缓冲放取共用 mutex；奇偶分别计产品许可。

```
semaphore empty=N, odd=0, even=0, mutex=1;
P1: while (true) {
    x=produce();
    P(empty); P(mutex); put(); V(mutex);
    if (x%2) V(odd); else V(even);
}
P2: while (true) {
    P(odd); P(mutex); getodd(); V(mutex);
    V(empty); countodd();
}
P3: while (true) {
    P(even); P(mutex); geteven(); V(mutex);
    V(empty); counteven();
}
```

put() 放入 P1 当前的本地整数 x。实际取出后立即归还空位，私有统计留在锁外；某类长期不产生时，该消费者可一直等，其他进程仍可推进。

例 2-13　empty=1000，full=0，mutex=batch=1

mutex 保护单次缓冲操作及指针，batch 保证一个消费者连续取 10 件；生产者不申请 batch。

```
semaphore empty=1000, full=0, mutex=1, batch=1;
item buf[1000]; int in=0, out=0;
producer: while (true) {
    x=produce(); P(empty); P(mutex);
    buf[in]=x; in=(in+1)%1000;
    V(mutex); V(full);
}
consumer: while (true) {
    P(batch);
    for (int k=0; k<10; k++) {
        P(full); P(mutex);
        local[k]=buf[out]; out=(out+1)%1000;
        V(mutex); V(empty);
    }
    V(batch); consume(local);
}
```

消费者缺货时只持 batch、等待 full，生产者仍可取得 mutex 补货；其他消费者无法插入。每取一件即归还空位，整批无须在开始时凑齐。

例 2-14　D

54. D

“条件变量”是管程内部说明和使用的一种特殊变量，其作用类似于信号量机制中的“信号量”，都用于实现进程同步。需要注意的是，在同一时刻，管程中只能有一个进程在执行。若进程 A 执行了 x.wait()操作，则该进程会阻塞，并挂到条件变量 x 对应的阻塞队列上。这样，管程的使用权被释放，就可以有另一个进程进入管程。若进程 B 执行了 x.signal()操作，则会唤醒 x 对应的阻塞队列的队首进程。只有一个进程要离开管程时才能调用 signal()操作。

例 2-15 B

39. B

死锁的处理采用三种策略：死锁预防、死锁避免、死锁检测和解除。

死锁预防采用破坏产生死锁的 4 个必要条件中的一个或几个来防止发生死锁。其中之一的“破坏循环等待条件”，一般采用顺序资源分配法，即限制了用户申请资源的顺序，因此说法 I 的前半句属于死锁预防的范畴。此外，银行家算法虽然会通过检测是否存在安全序列来判断申请资源的请求是否合法，但安全序列并不是唯一的，也不是固定的，它只是一种可能的分配方案，而不是一种必须遵循的规则，银行家算法更没有给出固定的申请资源的顺序，因此说法 I 错误。

银行家算法是最著名的死锁避免算法，其中的最大需求矩阵 Max 定义了每个进程对 m 类资源的最大需求量，系统在执行安全性算法中都会检查此次资源试分配后，系统是否处于安全状态，若不安全则将本次的试探分配作废。在死锁的检测和解除中，系统为进程分配资源时不采取任何措施，但提供死锁检测和解除的手段，一旦检测到系统发生死锁，就立即采取相应的措施来解除死锁，因此不用关心进程所需的总资源量。说法 II、III 正确。

例 2-16 D；不存在安全序列

Work 初为 $(0,2,1)$，仅 P1 可完成，回收后为 $(2,2,1)$；再选 P4 后为 $(2,2,2)$。P2、P3 均仍需 3 个 R2，仅 2 个可用，无法继续。完成前两人无法证明整个系统安全；实际当前请求未给出，不能据此确定已死锁。

例 2-17 C；仅 I

26. C

$$\text{Need} = \text{Max} - \text{Allocation} = \begin{bmatrix} 0 & 0 & 4 \\ 1 & 7 & 5 \\ 2 & 3 & 5 \\ 0 & 6 & 4 \\ 0 & 6 & 5 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 3 \\ 1 & 0 & 0 \\ 1 & 3 & 5 \\ 0 & 0 & 2 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 7 & 5 \\ 1 & 0 & 0 \\ 0 & 6 & 2 \\ 0 & 6 & 4 \end{bmatrix}$$

说法 I：根据 need 矩阵可知，当 Available 为(1, 4, 0)时，可满足 P_2 的需求；P_2 结束后释放资源，Available 为(2, 7, 5)可以满足 P_0, P_1, P_3, P_4 中任意一个进程的需求，所以系统不会出现死锁，处于安全状态。说法 II：当 Available 为(0, 6, 2)时，可以满足进程 P_0, P_3 的需求；这两个进程结束后释放资源，Available 为(0, 6, 7)，仅可以满足进程 4 的需求；P_4 结束并释放后，Available 为(0, 6, 8)，此时不能满足余下任意一个进程的需求，因此当前处在非安全状态。说法 III：当 Available 为(1, 1, 1)时，可以满足进程 P_0, P_2 的需求；这两个进程结束后释放资源，Available 为(2, 4, 9)，此时不能满足余下任意一个进程的需求，处于非安全状态。说法 IV：当 Available 为(0, 4, 7)时，可以满足 P_0 的需求，进程结束后释放资源，Available 为(0, 4, 10)，此时不能满足余下任意一个进程的需求，处于非安全状态。综上分析：只有说法 I 处于安全状态。

内存管理

例 3-1 C

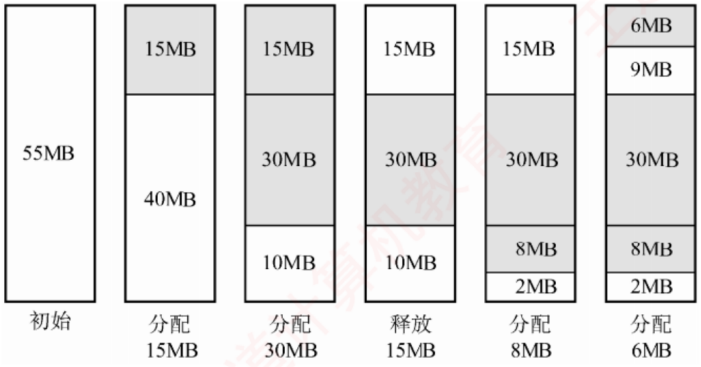
71. C

进程 R 和 S 共享数据 data，说明它们都映射了同一个共享内存段，于是这个段在物理内存中的位置（页框号）必然相同，但这两个段在不同进程的地址空间中的位置（页号）可以不同。因此，p1 和 p2 不一定相等，f1 和 f2 一定相等。

例 3-2 B；9 MB

61. B

最佳适配算法是指每次为作业分配内存空间时，总是找到能满足空间大小需要的最小空闲分区给作业，可以产生最小的内存空闲分区。下图显示了这个过程的主存空间变化。



图中，灰色部分为分配出去的空间，白色部分为空闲区。这样，容易发现，此时主存中最大空闲分区的大小为 9MB。

例 3-3 B；128

按原解每张下级页表占一页，可放 $2^{10}/2 = 2^9$ 项；覆盖 2^{16} 页需 $2^{16}/2^9 = 2^7 = 128$ 个目录项。字段为 7 位目录、9 位页号、10 位偏移。该页表按页存放是原解采用的布局条件。

例 3-4 A；081H、101H

偏移为低 12 位 225H；目录号 $VA \gg 22 = 081_H$；下级页号 $(VA \gg 12)\ \&\ 3FF_H = 101_H$。10 位字段须按二进制位宽提取。

例 3-5 C；5 次

按原解页框初始空，LRU 从最久到最近记录：

引用	更新后次序	结果
0、1、2、7	0、1、2、7	4 次初装
0	1、2、7、0	命中
5	2、7、0、5	淘汰 1
3	7、0、5、3	淘汰 2
5	7、0、3、5	命中
0	7、3、5、0	命中

引用	更新后次序	结果
2	3、5、0、2	淘汰 7
7	5、0、2、7	淘汰 3
6	0、2、7、6	淘汰 5

9 次缺页减 4 次空框装入，共 5 次置换。

例 3-6 C；60A01H

页号 2、偏移 A01H，存在位 0，须调页。固定两框已满，页 3、4 的状态为 $(1,0)$、$(1,1)$，改进 Clock 清访问位后先选干净的页 3。页 2 装入框 60H，$PA = 60_H \times 1000_H + A01_H = 60A01_H$；旧页框字段 20H 在存在位 0 时无效。

例 3-7 A；{6,0,3,2}

t 前最近六次引用为 6、0、3、2、3、2，去重得到 {6,0,3,2}。竖线右侧的 0 尚未访问。

例 3-8 210 ns；100000220 ns；110 ns；101565H

15.【解答】

1）根据页式管理的工作原理，应先考虑页面大小，以便将页号和页内位移分解出来。页面大小为 4KB，即 2^{12}，得到页内位移占虚地址的低 12 位，页号占剩余高位。可得三个虚地址的页号 P 如下（十六进制的一位数字转换成二进制的 4 位数字，因此十六进制的低三位正好为页内位移，最高位为页号）：

2362H：$P = 2$，访问快表 10ns，因初始为空，访问页表 100ns 得到页框号，合成物理地址后访问主存 100ns，共计 10ns + 100ns + 100ns = 210ns。

1565H：$P = 1$，访问快表 10ns，落空，访问页表 100ns 落空，进行缺页中断处理 10^8ns，访问快表 10ns，合成物理地址后访问主存 100ns，共计 10ns + 100ns + 10^8ns + 10ns + 100ns = 100000220ns。

25A5H：$P = 2$，访问快表，因第一次访问已将该页号放入快表，因此花费 10ns 便可合成物理地址，访问主存 100ns，共计 10ns + 100ns = 110ns。

2）当访问虚地址 1565H 时，产生缺页中断，合法驻留集为 2，必须从页表中淘汰一个页面，根据题目的置换算法，应淘汰 0 号页面，因此 1565H 的对应页框号为 101H。由此可得 1565H 的物理地址为 101565H。

例 3-9 10801008H；042H、001H；00201108H；00301004H

21.【解答】

1）

① 页面大小 = 2^{12}B = 4096B = 4KB。每个数组元素 4B，每个页面可以存放 4KB/4B = 1024 个数组元素，正好是数组的一行，数组 a 按行优先方式存放。1080 0000H 的虚页号为

10800H，因此 a[0]行存放在虚页号为 10800H 的页面中，a[1]行存放在页号为 10801H 的页面中。a[1][2]的虚拟地址为 10801 000H + 4×2 = 10801 008H。

② 转换为二进制 0001000010 0000000001 000000001000，根据虚拟地址结构可知，对应的页目录号为 042H，页号为 001H。

③ 进程的页目录表始址为 0020 1000H，每个页目录项长 4B，因此 042H 号页目录项的物理地址是 0020 1000H + 4×42H = 0020 1108H。

④ 页目录项存放的页框号为 00301H，二级页表的始址为 00301 000H，因此 a[1][2]所在页的页号为 001H，每个页表项 4B，因此对应的页表项物理地址是 00301 000H + 001H×4 = 00301 004H。

2）根据数组的随机存取特点，数组 a 在虚拟地址空间中所占的区域必须连续，因为数组 a 不止占用一页，相邻逻辑页在物理上不一定相邻，所以数组 a 在物理地址空间中所占的区域可以不连续。

3）由 1）可知每个页面正好可以存放一整行的数组元素，“按行优先方式存放”意味着数组的同一行的所有元素都存放在同一个页面中，同一列的各个元素都存放在不同的页面中，因此数组 a 按行遍历的局部性较好。

文件管理

例 4-1 B

67. B

当文件 F 首次被进程 P 打开时，会将磁盘的索引节点复制到内存的索引节点，并新增链接计数（打开计数）等内容。当进程 P 关闭文件 F 时，会将内存索引节点的链接计数减 1，因为文件 F 仅被进程 P 打开，所以链接计数变为 0，文件系统释放 F 的内存索引节点，选项 B 正确。关闭 F 不改变目录中文件 F 的目录项，选项 A 错误。关闭 F 只对内存索引节点进行相关的操作，不改变磁盘索引节点的数据结构，选项 C、D 错误。

例 4-2 D；20 位

60．D

需要注意的是，二进制位串表示的访问权限是这个文件所持有的，而不是某个用户所持有的，这个二进制位串要表示所有类别的用户的访问权限，每个要访问该文件的用户在访问之前都要查这个二进制位串。对于每类用户来说，都需要用 5 位来表示 5 种访问权限中的哪几种访问权限，共有四类用户，因此可将用户访问权限抽象为一个矩阵，其行代表用户类别，列代表访问权限。这个矩阵有 4 行 5 列，1 代表 true，0 代表 false，所以需要 20 位。

例如，下表是一个用二进制位串表示的文件权限。

用户类别	访问权限				
	完全控制	执　行	修　改	读　取	写　入
安全管理员	1	1	1	1	0
文件主	1	1	1	1	1
文件主的伙伴	0	1	0	1	0
其他用户	0	0	0	1	0

本题易误选 A，有读者认为用 2 位来表示用户类别，用 3 位来表示访问权限，这种情况在于没有理解文件访问权限的二进制位串是文件所持有的，所以当用户需要检查访问权限时，要在二进制位串中查询所属用户类别的访问权限。此外，每类用户的访问权限不是拥有 5 种访问权限中的哪一种，而是 5 种访问权限都可能拥有，因此不可能用 3 位来表示访问权限。

例 4-3 B；1、3

59．B

10 个直接索引指针指向的数据块大小为 10×1KB = 10KB。每个索引指针占 4B，则每个磁盘块可存放 1KB/4B = 256 个索引指针，一级索引指针指向的数据块大小为 256×1KB = 256KB，二级索引指针指向的数据块大小为 256×256×1KB = 2^{16}KB = 64MB。

按字节编址，偏移量为 1234 时，因 1234B < 10KB，由直接索引指针可得到其所在的磁盘块地址。文件的索引节点已在内存中，因此地址可直接得到，因此仅需 1 次访盘即可。

偏移量为 307400 时，因 10KB + 256KB < 307400B < 64MB，可知该偏移量的内容在二级索引指针所指向的某个磁盘块中，索引节点已在内存中，因此先访盘 2 次得到文件所在的磁盘块地址，再访盘 1 次即可读出内容，共需要 3 次访盘。

例 4-4 D；6 次

25．D

记录成组分解技术是指将若干逻辑记录存入一个块，一个逻辑记录不能跨越两个块。逻辑记录的固定长度为 100B，盘块长度为 512B，采用成组分解技术，因此每个盘块可以存放 5 个逻辑记录，剩下 12B 的空间可用于存放指向下一个盘块的指针。每个盘块存放 5 个逻辑记录，因此第 22 个逻辑记录在第 5 个物理块中（⌈22/5⌉= 5）。采用链接分配方式，要找到第 5 个物理块，需要从第 1 个物理块开始按指针顺序查找，因此需要启动磁盘 5 次。修改完后还要写回磁盘，因为此前已获得该块的磁盘地址，所以只需启动磁盘 1 次，共需启动磁盘 6 次。

例 4-5 B

61. B

多个进程打开一个文件时，读/写指针的位置不同，它们保存在各自的用户打开文件表中，说法 I 错误。硬链接是基于索引节点的共享方式，指向同一个内存索引节点，说法 II 正确。不同进程获得的文件描述符是各自独立的，分别指向各自的用户打开文件表中的一项，说法 III 正确。

例 4-6 C；82、1

14. C

盘块号 = 起始块号 +⌊盘块号/(1024×8)⌋= 32 +⌊409612/(1024×8)⌋= 32 + 50 = 82，这里问的是块内字节号而不是位号，因此还需要除以 8，块内字节号 =⌊(盘块号% (1024×8))/8⌋= 1。

例 4-7 （course,2)、(doc,10)；30；2 块；一级和二级

(1) stu 的两目录项为 `(course,2)`、`(doc,10)`，按图给定子项作答。

(2) doc 与 course1 同属该文件系统的 inode10，数据块号相同，x=30。

(3) course 目录内容已缓存，取得 inode 号后还须读 inode 所在块、数据块 30，共 2 块；题设未给这两者缓存。

(4) 6 MiB 需 1536 个 4 KiB 数据块；每索引块 1024 项，直接及一级共 1034 块，还差 502 块，须用一级及二级间接，无需三级。一级区用 1 个索引块；二级区用 1 个上层及 1 个下层索引块，索引开销另计。

例 4-8 $(8+1024+1024^2+1024^3)4096$ B；2^{26} 个；时间不同

(1) 每索引簇容纳 1024 个簇号，用户长度上限为 $(8+1024+1024^2+1024^3)\times 4096$ B，不含索引簇。

(2) 每 inode 簇放 64 个 inode，inode 区共 $2^{20}\times 64=2^{26}$ 个。5600 B 图像各占 2 数据簇，数据区可放 256M 个，均在直接区无需间接簇；取较小值 64M=67 108 864 个，按题给可用区不另假设目录占用。

(3) F1 末簇是逻辑簇 1，在直接区；F2 末簇是逻辑簇 9，需一级间接。inode 同已缓存、间接索引未缓存时，取簇号后者多读一次索引簇；若还读目标数据则分别 1、2 次读盘。

输入输出管理

例 5-1 C

40. C

计算柱面号、磁头号和扇区号的工作是由设备驱动程序完成的。题中的功能因设备硬件的不同而不同，因此应由厂家提供的设备驱动程序实现。

例 5-2 B；1550 μs、1100 μs

38. B

在单缓冲区中，当上一个磁盘块从缓冲区读入用户区完成时，下一磁盘块才能开始读入，也就是当最后一个磁盘块读入用户区完毕时所用的时间为 150×10 = 1500μs，加上处理最后一个磁盘块的时间 50μs，得 1550μs。双缓冲区中，不存在等待磁盘块从缓冲区读入用户区的问题，10 个磁盘块可以连续从外存读入主存缓冲区，加上将最后一个磁盘块从缓冲区送到用户区的传输时间 50μs 及处理时间 50μs，也就是 100×10 + 50 + 50 = 1100μs。

例 5-3 B；5046

16．B

每个柱面上有 8 个磁道（表示有 8 个磁头），每个磁道有 8 个扇区，因此每个柱面有 8×8＝64 个扇区。由题意可知，柱面号、盘面号、扇区号和逻辑记录编号都是从 0 开始的，因此 78 号柱面的 6 号磁道的 6 号扇区存放的是文件的第 78×64＋6×8＋6＝5046 个逻辑记录。

例 5-4 B

30．B

新磁盘是空白盘，必须分成扇区以便磁盘控制器能进行读/写操作，这个过程称为低级格式化（或物理格式化）。低级格式化为每个扇区使用特别的数据结构，说法 III 错误。为了使用磁盘存储文件，操作系统还需要将自己的数据结构记录在磁盘上。这分为两步。第一步是将磁盘分为由一个或多个柱面组成的分区，每个分区可以作为一个独立的磁盘，说法 I 错误。在分区之后，第二步是逻辑格式化（创建文件系统）。在这一步，操作系统将初始的文件系统数据结构存储到磁盘上。这些数据结构包括空闲和已分配的空间及一个初始为空的目录，说法 II、IV 正确。

例 5-5 C；325

29．C

SCAN 算法就是电梯调度算法。顾名思义，若开始时磁头向外移动，就一直要到最外侧，然后返回向内侧移动，就像电梯若往下，则一直要下到底层才会再上升一样。当前磁头位于 58 号并从外侧向内侧移动，先依次访问 130、180 和 199，然后返回向外侧移动，依次访问 42 和 15，因此磁头移过的磁道数是(199－58)＋(199－15)＝325。

例 5-6 ②→⑥→④→③→①→⑤；③ 为驱动；处理入口时 P 阻塞、CPU 内核态

六项次序为 ②→⑥→④→③→①→⑤：P 无字符可读先阻塞，键入后中断入口启动，驱动读字符至系统缓冲，唤醒 P，P 被调度后返回系统调用。

(1) ① 前 ③、后 ⑤，⑥ 后 ④，仅比较题列操作。(2) ② 后当前 P 必须让出 CPU，无其他就绪者时进 idle；① 后 P 恢复可调度资格。(3) ③ 执行具体控制器读入，属于驱动。(4) 入口及读字符阶段 P 仍阻塞，执行 ① 才就绪；CPU 执行中断处理在内核态，被中断者可能为另一进程。

例 5-7 307200000 B；100260→101660→110560→60005；(100,5,60)；驱动

(1) 容量 300 × 10 × 200 × 512 ＝ 307200000 B，即 300000 KiB。每簇 1024 B，每柱面 1000 簇。

(2) 请求柱面依次为 100、60、101、110。SSTF 从 85 先到 100，再到 101、110、60，对应簇 100260、101660、110560、60005；距离分别 15、1、9、50，共 75 柱面。每步按新磁头位置重算。

(3) 100530 ＝ 100 × 1000＋530，530 ＝ 5 × 100＋30，故柱面 100、磁头 5，磁道内第 30 簇从扇区 60 开始并含 61。起始 CHS 为 (100,5,60)，各字段 0 基。按本题传统几何模型由磁盘驱动转换。

计算机网络 4

计算机网络体系结构

例 1-1 B

29. B

在 OSI 参考模型中，传输层是自下而上第一个提供端到端服务的层，通过端口号实现应用进程间的逻辑通信。数据链路层仅负责相邻节点（如交换机、路由器等中间设备）之间的通信，而网络层提供主机到主机的通信，不提供面向应用进程的服务。

例 1-2 B

38. B

在 OSI 参考模型中，流量控制功能在多个层次存在：数据链路层实现相邻节点之间的流量控制，网络层关注整个网络中的拥塞与流量调节，传输层则提供端到端的流量控制。本题明确要求“两个相邻节点间”的流量控制，因此由数据链路层实现。

例 1-3 D（80.10 ms）

按题设分组长度直接切分文件，分组数为 $10^6/1000 = 1000$，每组发送时间为 $8000/10^8 = 80\ \mu s$。每段传播时间由时延带宽积求得 $1000/10^8 = 10\ \mu s$。

两段链路上第一组用 $2 \times 80 + 2 \times 10 = 180\ \mu s$ 到齐；后续 999 组每隔 80 μs 到齐。总计

$$180 + 999 \times 80 = 80100\ \mu s = 80.10\ \text{ms},$$

故选 **D**。本题按给定 1000 B 分组切分，不另添加未给定首部。

例 1-4 B

20. B

在建立虚电路的阶段，分组才需要携带完整的目的地址，以进行路由器选择。之后，每个分组只需携带短的虚电路号即可，属于同一条虚电路的分组按照同一路由进行转发，分组到达终点的顺序与发送顺序相同，可以保证有序传输。虚电路只是一条逻辑上的连接，并不是一条物理上的连接，因此不需要为每条虚电路预分配带宽。

物理层

例 2-1 C

26. C

在 ASK 调制中，4 个幅值表示 4 种信号状态（$V=4$），每个码元携带 $\log_2 4=2$ 比特。根据奈奎斯特定理，最大数据传输速率 $=2W\log_2 V$，已知带宽 $W=200\text{kHz}$，代入得 800kb/s。

例 2-2 A（约 2 倍）

17. A

本题给出了信道带宽、信噪比和编码方式，需要综合奈氏准则与香农定理的限制。在原始条件下：香农定理的极限速率为 $10\times\log_2 1001\approx 100\text{Mb/s}$（注意，这里的信噪比应使用无单位记法），奈式准则的极限速率为 $2\times10\times\log_2 32=100\text{Mb/s}$。带宽和信噪比提升后：香农定理的极限速率为 $20\times\log_2 10001\approx 260\text{Mb/s}$，奈式准则的极限速率为 $2\times20\times\log_2 32=200\text{Mb/s}$。由于实际速率受限于二者中的较小值，故提升后的极限速率为 200Mb/s，约为原来的 2 倍。

例 2-3 C

12. C

物理层接口规范涵盖四类特性：机械特性（如接口形状、尺寸）、电气特性（如信号电平、电压范围）、功能特性（如引脚功能、电平含义）和过程特性（事件发生顺序）。而物理地址（MAC 地址）用于标识数据链路层的设备，属于数据链路层的范畴，不在物理层定义范围内。

数据链路层

例 3-1 A

01. A

HDLC 协议对比特串组帧时，HDLC 数据帧以比特模式 0111 1110 标识每个帧的起始和结束，因此在帧数据中只要出现 5 个连续的位“1”，就在输出的位流中填充一个“0”。因此，组帧后的比特串为 01111100 00111110 10（下划线部分为新增的 0）。

例 3-2 D

08. D

观察选项：除后 4 位外，前 5 位都为 10111，可知发送方发送的数据部分为 10111，列式求得余数部分为 1100，因此发送方发送的帧串为 10111 1100。

```
          10100
10011 ) 101110000
        10011
        -----
          10000
          10011
          -----
           1100 ------余数
```

例 3-3 C

09. C

首先计算该编码集的码距：对四个码字两两比较，任意两个码字的码距至少为 4，故该编码集的码距 $d_{min}=4$。根据差错控制理论，若编码集的码距为 d，则可检测不超过 $d-1$ 位错误，能纠正不超过 $\lfloor (d-1)/2 \rfloor$ 位错误。因此该编码集最多能检测 3 位错误、纠正 1 位错误。

例 3-4 D

29. D

发送数据帧和确认帧的时间均为 $t=1000\times8b\div10kb/s=800ms$。

发送周期为 $T=800ms+200ms+800ms+200ms=2000ms$。

信道利用率为 $t/T\times100\%=800/2000=40\%$。

例 3-5 C

22. C

在 GBN 协议中，当接收方检测到某帧出错时，会简单地丢弃该帧及所有的后续帧，发送方超时后需重传该数据帧及所有的后续帧。注意，在 GBN 协议中，接收方一般采用累积确认的方式，即接收方对按序到达的最后一个分组发送确认，因此本题中收到 3 号帧的确认就表示编号为 0、1、2、3 的帧已接收，而此时发送方未收到 1 号帧的确认只能代表确认帧在返回的过程中丢失，而不代表 1 号帧未到达接收方。因此需要重传的帧为编号是 4、5、6、7 的帧。

例 3-6 C

19. C

为使信道利用率最高（100%），要让发送方在一个发送周期内持续发送帧，不能出现发送窗口内的帧发完但还未收到第一个帧的确认帧的情况。发送周期 = 发送一个数据帧的时间 + 往返时延 + 发送一个确认帧的时间，发送一个数据帧或确认帧的时间均为 $128B\div16kb/s=64ms$，发送周期 $=64ms+250ms\times2+64ms=628ms$。为保证发送方持续发送帧，在一个发送周期内至少要发送的帧数为 $628ms/64ms\approx10$，即发送窗口大小至少为 10，所以帧序号至少采用 4 比特。

例 3-7 B

21. B

将收到的序列每 4 个数分成一组，即(2, 0, 2, 0), (0, −2, 0, −2), (0, 2, 0, 2)，因为题目求的是 A 站发送的数据，因此将这三组数据与 A 站的码片序列(1, 1, 1, 1)做内积运算，结果分别是(2, 0, 2, 0)•(1, 1, 1, 1)/4 = 1，(0, −2, 0, −2)•(1, 1, 1, 1)/4 = −1，(0, 2, 0, 2)•(1, 1, 1, 1)/4 = 1，所以 C 站接收到的 A 站发送的数据是 101。

例 3-8 D

37. D

有关最短帧长的题要抓住两个公式来分析：① 发送帧的时间≥争用期的时间；② 最短帧长=数据传输速率×争用期的时间。题中，最短帧长减少 800 比特，则发送帧的时间减少 0.8μs，要使①和②依然成立，就需要至少将争用期（信号的往返时间）的时间减少 0.8μs，所以往返传播的总距离至少需要减少 200000km/s×0.8μs = 160m，即单程距离至少需要减少 80m。

例 3-9 B

48. B

数据帧的长度为 1998B，链路带宽为 54Mb/s，因此数据帧的发送时延为 1998B÷54Mb/s = 296μs。网络分配向量（NAV）指出了信道忙的持续时间，含义是“正在通信的两个站点以外的站点都不能在这段时间内发送数据”。CSMA/CA 协议中的 RTS 帧、CTS 帧和数据帧都携带占用信道的持续时间，当 A 站广播一个 RTS 帧时，将占用信道的持续时间（SIFS + CTS + SIFS + DATA + SIFS + ACK）写入 RTS 帧的首部；当 AP 收到 RTS 帧后，广播一个 CTS 帧，将占用信道的持续时间（SIFS + DATA + SIFS + ACK）写入 CTS 帧的首部；之后传送的数据帧的首部也携带本次通信所需的持续时间。其他站收到这些帧后，根据帧中的持续时间设置自己的 NAV 值，因此隐蔽站 B 收到 AP 发送的 CTS 帧时，设置自己的 NAV 值为 SIFS + DATA + SIFS + ACK = 28μs + 296μs + 28μs + 2μs = 354μs。

例 3-10 A

08. A

以太网 MAC 帧的最小帧长为 64B，数据字段的长度至少为 46B，但 802.1Q 帧会额外插入 4B 的 VLAN 标签，所以 802.1Q 帧的数据字段的长度至少为 42B，因此需要额外填充 2 字节。

例 3-11 B

20. B。

直通交换方式的输入接口接收到一个帧时，只检查帧的目的 MAC 地址决定输出接口，引入的转发时延至少为读取目的 MAC 地址所需的时间。目的 MAC 地址共 6B，引入的转发时延至少为 6×8bit÷100Mb/s = 0.48μs。存储转发方式引入的转发时延则至少为读取整个帧的时间。

例 3-12　（1）3 帧；（2）5 帧，首 $S_{5,2}$、末 $S_{1,2}$；（3）3 帧，首 $S_{2,3}$；（4）50%。

读题时先分清两条状态线：收到帧的确认序号决定 “我发出的哪些帧已被确认”；收到帧的发送序号决定 “我下次应捎带哪个确认号”。GBN 的接收窗口为 1，收到失序帧不能跳过缺口向前确认。序号空间为 $2^3 = 8$，发送窗口最多为 7，所有序号按模 8 回绕。

图 (a) 最后收到 $R_{3,3}$。其确认号 3 累计确认甲方序号 0、1、2 已正确到达，因此第（1）问为 3 帧：$S_{0,0}$、$S_{1,0}$、$S_{2,0}$。不能因它的发送序号为 3，便推断甲方也收齐了乙方 0～3 号帧：甲方依次正确收到 $R_{0,1}$、$R_{1,3}$，随后缺少乙方 2 号帧，$R_{3,3}$ 作为失序帧被丢弃，所以甲方仍希望收到乙方 2 号帧。

此时甲方已发送到 $S_{4,1}$，未确认的是发送序号 3、4 两帧。7 个窗口位置中还剩 $7-2=5$ 个；下一发送序号从 5 开始，捎带确认号始终为 2，故第（2）问可发送 $S_{5,2}$、$S_{6,2}$、$S_{7,2}$、$S_{0,2}$、$S_{1,2}$，首帧为 $S_{5,2}$，末帧为 $S_{1,2}$。回绕后的 0、1 是新一轮数据，不能与已经确认的旧 0、1 混为一谈。

图 (b) 乙方最后通告确认号 2，意味着甲方 0、1 号已确认，2、3、4 号仍未确认。$S_{2,0}$ 超时触发从最早未确认帧开始的后退重传，因此第（3）问重发 3 帧。甲方此时已按序收到乙方 $R_{2,2}$，期待乙方下一帧为 3，故重传首帧应更新捎带确认号，写成 $S_{2,3}$；随后为 $S_{3,3}$、$S_{4,3}$，不能照抄旧帧的确认号 0。

第（4）问须保留捎带确认帧的发送时间。发送一个数据帧需 $T_d = 1000 \times 8/(100 \times 10^6) = 0.08$ ms；确认也装在 1000 B 的数据帧中，$T_a = 0.08$ ms。沿用教材模型：题目中的 RTT 表示往返传播时延；从发送首帧到完整收到其确认，周期为 $T_d + \mathrm{RTT} + T_a = 1.12$ ms。最多连续发 7 帧，故 $U_{\max} = 7 \times 0.08/1.12 = 50\%$。全双工允许两个方向同时工作，但本题求甲方单方向利用率；也不能把长捎带确认当成长度可忽略的纯 ACK。

网络层

例 4-1　D

对 Net1，经 A、B、C、D 的总距离为 9、33、32、28，取 9；Net2 为 20、45、42、34，取 20；Net3 为 32、28、28、42，取 28；Net4 为 44、40、20、30，取 20。因此选 **D**。Net3 经 B、C 并列最优，不影响距离答案；若另问下一跳，就须保留并列或按指定规则选择。

例 4-2　B

66．B

链路层 MTU＝800B。IP 分组首部长 20B。片偏移以 8 个字节为偏移单位，因此除了最后一个分片，其他每个分片的数据部分长度都是 8B 的整数倍。所以，最大 IP 分片的数据部分长度为 776B。在总长度为 1580B 的 IP 数据报中，数据部分占 1560B，1560B/776B＝2.01…，需要分成 3 片。所以第 2 个分片的总长度字段为 796，MF 为 1（表示还有后续的分片）。

例 4-3　B

70．B

网络号位数为 20＝8×2＋4，子网掩码为 11111111 11111111 11110000 00000000，将它与主机地址 168.16.84.24 进行逐位“与”，得到网络地址为 168.16.80.0/20，该网段共有 $2^{12}-2$ 个可供分配的 IP 地址，地址范围是 168.16.80.1～168.16.95.254。

例 4-4 B

64. B

网络前缀为 20 位，将 101.200.16.0/20 划分为 5 个子网，为了保证有子网的可分配 IP 地址数尽可能小，即要让其他子网的可分配 IP 地址数尽可能大，不能采用平均划分的方法，而要采用变长的子网划分方法，也就是最大子网用 1 位子网号，第二大子网用 2 位子网号，以此类推。

子网 1：101.200.00010000.00000001～101.200.00010111.11111110；地址范围为 101.200.16.1/21～101.200.23.254/21；可分配的 IP 地址数为 2046 个。

子网 2：101.200.00011000.00000001～101.200.00011011.11111110；地址范围为 101.200.24.1/22～101.200.27.254/22；可分配的 IP 地址数为 1022 个。

子网 3：101.200.00011100.00000001～101.200.00011101.11111110；地址范围为 101.200.28.1/23～101.200.29.254/23；可分配的 IP 地址数为 510 个。

子网 4：101.200.00011110.00000001～101.200.00011110.11111110；地址范围为 101.200.30.1/24～101.200.30.254/24；可分配的 IP 地址数为 254 个。

子网 5：101.200.00011111.00000001～101.200.00011111.11111110；地址范围为 101.200.31.1/24～101.200.31.254/24；可分配的 IP 地址数为 254 个。

综上所述，可能的最小子网的可分配 IP 地址数是 254 个。

例 4-5 C

62. C

对于此类题目，先分析需要聚合的 IP 地址。观察发现，题中的 4 个路由地址，前 16 位完全相同，不同之处在于第 3 段的 8 位中，将这 8 位展开写成二进制，分别如下：

	7	6	5	4	3	2	1	0
32	0	0	1	0	0	0	0	0
40	0	0	1	0	1	0	0	0
48	0	0	1	1	0	0	0	0
56	0	0	1	1	1	0	0	0

观察发现，在 4 个地址的第 3 段中，从前向后最多有 3 位相同，因此这 3 位是能聚合的最大位数。将这些相同的位都保留，将第 3 段第 3 位后的所有位都置 0，就得到了聚合后的 IP 地址：35.230.32.0，其网络前缀为 16+3，即前 19 位，因此聚合后的网络地址为 35.230.32.0/19。

例 4-6 B

43. B

先判断乙和甲是否在同一个子网内，用 IP 地址和子网掩码进行按位“与”运算，然后检查它们的网络前缀是否相同。若乙和甲在同一个子网内，则直接交付，无须经过网关，因此将乙的 MAC 地址作为帧的目的 MAC 地址。若乙和甲不在同一个子网内，则要通过网关转发，因此将网关的 MAC 地址作为帧的目的 MAC 地址。经计算，甲的网络前缀为 211.71.128.0/20，乙的网络前缀为 211.71.128.0/20，乙和甲在同一个子网内，因此帧的目的 MAC 地址为乙的 MAC 地址。

例 4-7 C

H 仅收到地址提议，尚未获得 192.168.5.9 的使用确认。初始选择阶段 Request 使用源 0.0.0.0、目的 255.255.255.255，按“目的、源”的设问顺序选 **C**。服务器的单播地址不是这一步的目的 IP 地址。

图中 Request 框沿用原题的 `yiaddr` 标注。按 DHCP 规范，客户端此阶段申请的地址实际由 Requested IP Address 选项携带，客户端发出的 `yiaddr` 为 0。图中的 192.168.5.9 应理解为待申请地址信息，不能据这个标注推出 H 已经能用它作 IP 源地址。Offer 中的 `yiaddr` 才是服务器提出的地址。

例 4-8 C

51. C

ICMP 差错报告报文有 5 种：终点不可达、源点抑制、时间超过、参数问题、改变路由。其中源点抑制是指当路由器或主机因拥塞而丢弃数据报时，向源点发送源点抑制报文，使源点知道应当把数据报的发送速率放慢。注意，最新的 ICMP 标准已不再使用源点抑制报文。

例 4-9 D

07. D

IPv4 地址占 32 位，地址空间为 2^{32}；IPv6 地址占 128 位，地址空间为 2^{128}，IPv6 地址空间是 IPv4 地址空间的 2^{96} 倍，说法 I 错误。IPv4 首部长度是 4B 的倍数，长度可变；IPv6 基本首部长度是 40B，不可变，说法 II 错误。IPv4 向 IPv6 过渡可以采用双协议栈（设备同时支持 IPv4 和 IPv6）和隧道技术（IPv6 数据报封装 IPv4 的数据部分），说法 III 正确。IPv6 首部的 Hop Limit 字段和 IPv4 首部的 TTL 字段都用于限制数据报在网络中经过的路由器数量，说法 IV 正确。

例 4-10 （1）192.168.1.127，192.168.1.128/25，45 台；（2）2 片，796 B 与 724 B。

“均分一个`/24`”先确定两个`/25`，每个块有 128 个地址，再利用已经分配的地址识别哪一块属于哪个部门。销售部`.1～.20` 在 192.168.1.0/25 内，广播地址为 **192.168.1.127**；技术部`.129～.208` 在 **192.168.1.128/25** 内。技术部可用地址有 $128-2=126$ 个，已分配主机数为 $208-129+1=80$，F1 的`.254` 还占 1 个，因此还可连接 $126-80-1=$ **45** 台主机。路由器接口不是额外的一台用户主机，但会占用同一子网的可用地址。

分片计算从原 IP 数据部分开始：$1500-20=1480$ B。F1 所在链路每片总长不能超过 800 B，扣首部后最多 780 B；非末片数据长度又必须是 8 B 的整数倍，故最大数据量为 $\lfloor 780/8 \rfloor \times 8 =$ **776** B。余下 $1480-776=704$ B，一片即可容纳，所以最少 2 片。

分片	数据长度/B	IP 总长度/B	片偏移（8 B 单位）与 MF
第一片	776	796	偏移 0，MF=1
第二片	704	724	偏移 97，MF=0

第二片偏移是 $776/8=97$，它累计的是原数据的位置，不加第一片首部。末片不要求数据长度继续凑满 776 B，所有片的数据长度相加仍为 1480 B。

例 4-11 3 条路由；目的 194.17.20.200 走 E0；域间 BGP，由 TCP 承载。

先按出口分组，再分别求可用汇总前缀。经 S0 到达的 153.14.5.0/25、153.14.5.128/25 恰好拼成 153.14.5.0/24，下一跳为 R1 的 153.14.3.2。经 S1 到达的 194.17.20.0/25 与 194.17.21.0/24 可用 194.17.20.0/23 覆盖，但这个`/23` 也包含 E0 直连的 194.17.20.128/25，必须保留该更具体条目，让最长前缀匹配处理例外。

目的网络	下一跳	接口
153.14.5.0/24	153.14.3.2	S0

目的网络	下一跳	接口
194.17.20.0/23	194.17.24.2	S1
194.17.20.128/25	—（直连）	E0

这 3 条分别承担三个不同出口，已不能在保持全部转发结果的前提下继续合并。第二条覆盖较宽范围，E0 的更具体项处理直连例外。若删除 E0 例外，原本应直达的分组就会误发给 R3。

194.17.20.200 同时匹配 `194.17.20.0/23` 和 `194.17.20.128/25`；比较前缀长度 25 与 23，选择 **E0**，不依赖表项书写顺序。R1 与 R2 属于不同自治系统，因此交换域间路由使用 **BGP（BGP-4）**，BGP 报文由 **TCP** 承载。自治系统内部使用何种 IGP，不改变这条跨 AS 邻接的协议选择。

例 4-12 （1）OSPF；（2）16；（3）30 s、60 s；（4）eBGP/UPDATE、iBGP；（5）R14 选 R11，R15 选 R13。

第（1）问先检查协议的适用上限。RIP 最大有效跳数为 15，16 即不可达，AS4 的规模可能超出这一限制，应选 **OSPF**。第（2）问问的是 IP 的 TTL 而非 RIP 度量：每经过一个路由器 TTL 减 1，减至 0 便被丢弃。最多经过 15 个路由器后仍须至少剩 1，故初始 TTL 至少为 **16**。

第（3）问把每次交换视作一轮：本轮只能把交换前已知的路由传给相邻路由器，不能让新收到的信息在同一轮连续穿越全网。又因为首次交换定义为 $t=0$，直连路由在此刻已经传给一跳邻居，不能额外再计一个 30 s。

距首次交换的时间	已正确获知 210.2.3.0/24 的路由器	已正确获知 210.2.4.0/24 的路由器
0 s	R14、R11、R15	R16、R15
30 s	R11～R16 全部	R16、R15、R14、R13
60 s	R11～R16 全部	R11～R16 全部

210.2.3.0/24 直连 R14，首次传给 R11、R15，下一轮由它们传给 R12、R13、R16，所以需 **30 s**。210.2.4.0/24 直连 R16，首次仅传给 R15，30 s 时到 R14、R13，60 s 时才到 R11、R12，所以需 **60 s**。逐轮集合依赖题图中实际存在的边，不能擅自补上 R13—R16。

第（4）问按会话两端是否同属一个 AS 区分：R44 与 R13 分属 AS4、AS1，用 **eBGP** 会话，通过 **UPDATE** 报文通告；R13 向同属 AS1 的 R14、R15 传播可达性信息，用 **iBGP** 会话。BGP 会话不要求所有对端在物理上直接相连。

第（5）问按本题无策略约束的教材选路模型，先比较 AS_PATH：经 R11、R13 的路径各含 3 个 AS，经 R12 含 4 个，先排除 R12。在其余属性相同的候选中，再比较 AS 内到出口的距离。R14 到 R11 为 1 跳，到 R13 需经 R15 为 2 跳，选 **R11**；R15 到 R13 为 1 跳，到 R11 为 2 跳，选 **R13**。真实 BGP 还受本地优先级等属性影响，本题给定条件并不支持把 “AS 数少、出口近” 当成无条件覆盖所有策略的规则。

例 4-13 （1）0.24 s、200 kb/s、理想下界 0.40 s；（2）8 帧、4 bit；（3）作业区 10.10.10.64/26，管理区 10.10.10.0/26，生活区 10.10.10.128/25。

单向路径要经过 “地面 $\rightarrow$ 卫星 $\rightarrow$ 地面” 两段。按本题采用的几何模型，每段链路的长度取卫星高度，单向传播时延 $T_p = 2 \times 36000/300000 = \mathbf{0.24\ s}$。路径吞吐量受最慢链路限制，千兆以太网不能抬高 200 kb/s 的卫星瓶颈，因此 H 单方向最大吞吐量为 **200 kb/s**；全双工表示反方向另有同样容量，不能在一次上传中相加成 400 kb/s。

4000 B 经卫星链路发送需 $4000 \times 8/200000 = 0.16$ s。首部忽略时，最后一比特从卫星链路发送端到接收端还要传播 0.24 s，故教材模型给出的最短上传时间为 $0.16 + 0.24 = \mathbf{0.40\ s}$。这个 400 ms 还额外忽略了局域网的发送开销；题干给出的忽略项并不包含千兆链路上的发送与分帧，若要严格计入每段链路上的发送、分帧及存储转发，还需题目进一步指定相应条件。例如 4000 B 在一条 1 Gb/s 链路上仅串行发送也需 32 μs。因此 400 ms 适合作为本题理想下界，不能视为所有实际以太网转发方式下的精确完成时间。

SLP 每帧发送时延 $T_d = 1500 \times 8/200000 = 60$ ms；ACK 长度忽略，从开始发送首帧到收到其确认需 $60 + 2 \times 240 = 540$ ms。设窗口为 W，利用率 $U = \min(1, 60W/540)$；要求 $U \geq 0.8$，得到 $W \geq 7.2$，向上取整为 **8 帧**。GBN 接收窗口为 1，需 $2^k \geq W + 1 = 9$，所以序号字段最少 **4 bit**。3 bit 虽然有 8 个序号，却只能支持最大 7 帧的 GBN 发送窗口；其利用率 $7/9 \approx 77.78\%$，达不到要求。

最后按可用主机数决定块大小。生活区需 $2^7-2=126$ 个可用地址，至少占/25；另两区各需 $2^6-2=62$ 个，至少占/26。三块恰好用完 $128+64+64=256$ 个地址。H 的 10.10.10.33/26 锁定管理区为 10.10.10.0/26；生活区的/25 必须按 128 地址边界对齐，且不能与管理区重叠，只能取 10.10.10.128/25，留下 10.10.10.64/26 给作业区。

子网	CIDR 地址	可用地址范围	可用数量
作业区	**10.10.10.64/26**	10.10.10.65～10.10.10.126	62
管理区	**10.10.10.0/26**	10.10.10.1～10.10.10.62	62
生活区	**10.10.10.128/25**	10.10.10.129～10.10.10.254	126

传输层

例 5-1 B

15. B

传输层的分用是指接收方根据报文首部信息将数据交付给正确的应用进程。UDP 使用目的端口号标识目标进程，是分用的关键依据。源端口号主要用于回信，非分用所必需；长度和检验和字段不参与进程寻址。

例 5-2 C

16. C

UDP 检验和的计算方法是二进制反码求和再取反，二进制反码求和的运算规则：① 从低位到高位逐列进行加法运算，0 和 0 相加得 0，0 和 1 相加得 1，1 和 1 相加得 0 并产生进位 1；② 若最高位相加后产生进位，则将该进位加到结果的最低位，该过程也称回卷。计算过程如下：

```
  1011 1001 1011 0110
+ 0110 0101 1100 0101
-------------------------
[1] 0001 1111 0111 1011   ——→ 最高位产生了进位，最低位要加1
+                    1
-------------------------
  0001 1111 0111 1100     ——→ 最后一步要取反
  1110 0000 1000 0011
```

例 5-3 D

41. D

确认序号表示接收方期望收到的下一个字节的序号。第一个段起始序号为 200，包含 300B 数据，序号范围 200～499；第二个段起始序号为 500，包含 500B 数据，序号范围 500～999。因此，乙正确接收后，期望的下一个字节序号为 1000，故返回的确认序号为 1000。

例 5-4 D

58．D

题目问的是最少时间，因此当服务器 S 收到客户 C 发的 FIN 后不再传输数据，而立即返回 ACK 和 FIN。客户 C 主动关闭：C 发 FIN（0ms）→S 在 0.5RTT（25ms）收到后立即返回 ACK + FIN（因无数据待发）→C 在 1RTT（50ms）收到 S 的 ACK + FIN 后，发送 ACK 并启动 2MSL 计时器。C 进入 CLOSED 状态至少需 50ms + 2×800ms = 1650ms。S 在收到 C 对 FIN 的 ACK 后即进入 CLOSED 状态，该 ACK 在 C 发 FIN 后 1.5RTT = 75ms 到达 S，故 S 至少需 75ms。

例 5-5 D

52．D

由于慢开始门限 ssthresh 可按需设置，为求拥塞窗口 cwnd 从 8KB 增长到 32KB 所需的最长时间，可将 ssthresh 设置小于或等于 8KB。即始终处于拥塞避免阶段，cwnd 从 8KB 增至 32KB，需增加 24KB，每经过一个 RTT，cwnd 加 1，故需 24 个 RTT，总时间为 24×2ms = 48ms。

例 5-6 A

第一段 seq=2001 覆盖 2001～3000，第二段 seq=3001 覆盖 3001～4000。返回 ack_seq=3001 只累计确认第一段，第二段的 1000 B 仍在途。乙段的 seq=4001 属于乙 → 甲方向，不能拿它确认甲发出的字节。

甲原 cwnd 为 2000 B，小于门限 8000 B，收到一份对新数据的确认后，按题设慢开始增加 1 MSS，cwnd 变为 3000 B；rwnd 通告为 4000 B，总发送额度取 3000 B。减去在途 1000 B，尚可新发 2000 B，即 2 段，选 **A**。对应允许的甲方发送范围从 3001 起共 3000 B，其中 3001～4000 已经发出，4001～6000 还可新发。

例 5-7 A

47．A

t 时刻发生超时，将慢开始门限 ssthresh 设为 8KB/2 = 4KB，并将拥塞窗口 cwnd 重置为 1KB。接收窗口恒为 10KB，因此发送窗口 = min{cwnd, 10KB}。经历 10RTT 后，拥塞窗口的大小（单位为 KB）依次为 2, 4, 5, 6, 7, 8, 9, 10, 11, 12。由于接收窗口限制为 10KB，最终发送窗口为 10KB。实际上，由于发送窗口必然总小于或等于 10KB，四个选项中只有选项 A 符合条件。

例 5-8 （1）H 发送 1、3、4；握手 1、2、3；填充 3、5；（2）16 B；（3）15 个路由器。

按 “IP 长度 →TCP 起点 → 控制位 → 序号确认号” 顺序解码。所有分组首字节 45 表示 IPv4、IHL=5，即 20 B IP 首部，TCP 从第 21 字节开始。1、3、4 号的源地址 `c0 a8 00 08` 为 192.168.0.8，因此 **H 发送的是 1、3、4 号**；2、5 号的源地址为 S。

编号	IP 总长/B	TCP 首长/B	TCP 标志
1	48	28	SYN
2	48	28	SYN、ACK
3	40	20	ACK
4	56	20	PSH、ACK
5	40	20	ACK

编号	seq（十六进制）	ack（十六进制）	应用数据/B
1	846b41c5	00000000（ACK=0）	0
2	e0599fef	846b41c6	0
3	846b41c6	e0599ff0	0
4	846b41c6	e0599ff0	16
5	e0599ff0	846b41d6	0

1 号的 TCP 数据偏移高半字节为 7，首部长 $7\times4=28$ B；控制字节 02 置 SYN。2 号的控制字节 12 同时置了 SYN 与 ACK，确认 H 的初始序号加 1；3 号的控制字节 10 只置 ACK，确认 S 的初始序号加 1。因此 **1、2、3 号完成三次握手**，SYN 分别消耗一个序号，第三次纯 ACK 不额外消耗序号。捕获表只给前 40 B，1、2 号的 8 B TCP 选项没有完整列出，但长度字段足以判定它们没有应用数据。

快速以太网沿用无标签以太网数据字段至少 46 B 的限制，判断填充应比较**整个 IP 分组**与 46 B，不能只比较 TCP 数据。3、5 号 IP 总长 `0028` 为 40 B，均需补 $46-40=6$ B；1、2 号为 48 B、4 号为 56 B，均不需补。因此填充的是 **3、5 号**，每个形成 $14+40+6+4=64$ B 的最短 MAC 帧。填充不计入 IP 总长度，更不占 TCP 字节序号。

第(2)问利用累计确认而不是把 40 字节截取范围当作完整数据。H 首个应用字节的序号为 SYN 序号加 1，即 `846b41c6`；5 号通告下一个期望序号 `846b41d6`，所以 S 已按序收到 $\texttt{846b41d6}-\texttt{846b41c6}=\texttt{10}=\mathbf{16}$ B 应用数据。这也与 4 号的 $56-20-20=16$ B 相符。

第（3）问先匹配报文身份。S 行 IP 标识为 `6811`，TCP 序号、确认号与 5 号一致，对应 H 处的 5 号；途中的 NAT 可能改变目的地址与端口，不能因此误认为是另一个报文。S 发出时 TTL 为 `40` 即 64，H 处为 `31` 即 49，每经过一个路由器减 1，故经过 $64-49=\mathbf{15}$ 个路由器。交换机不减 TTL。图中的其余校验和字节按来源保留；题目只截取了部分字节，选项与有效载荷并未给全，不能据此声称已经验证所有报文的完整校验和。

例 5-9 （1）SYN=ACK=1，ack=101；（2）rwnd=12 KB，cwnd=9 KB，发送窗口 9 KB；（3）20581，163.84 kb/s；（4）300 ms。

第（1）问的第二次握手要同时表达 “接受对方 SYN” 和 “发出本方 SYN”，所以 **SYN=1、ACK=1、确认号=101**。初始序号 100 被 SYN 占用，首个数据字节从 101 开始。

第（2）问必须分别更新两种窗口。沿用教材模型：初始拥塞窗口为 1 MSS，每个新数据段的 ACK 使慢开始窗口增加 1 MSS。第 8 个数据确认到达后，$cwnd=(1+8)\times1=$ **9 KB**，仍小于 32 KB 阈值；S 已经缓存 8 KB 且未取出，通告 $rwnd=20-8=$ **12 KB**。发送窗口总额度为 $\min(cwnd,rwnd)=$ **9 KB**。如果题目问此时能再发送多少，还须扣除已经发出但未确认的数据，不能把 9 KB 直接当成新增发送量。

第（3）问要计到零窗口通告回到 H3 的时刻。每轮受拥塞窗口与剩余接收缓存共同限制，不能把慢开始序列无限写成 1、2、4、8、16。

轮次	发送前 cwnd/MSS	尚余接收缓存/KB	本轮发送/KB
1	1	20	1
2	2	19	2
3	4	17	4
4	8	13	8
5	16	5	5

轮次	累计已接收/KB	本轮确认到达时刻
1	1	1 RTT
2	3	2 RTT
3	7	3 RTT
4	15	4 RTT
5	20	5 RTT

第 5 轮只能再放入 5 KB, S 随后通告零窗口。下一个待发送字节的序号为 $101+20\times1024=$ **20581**, 这个值不因窗口暂时关闭而重置。计时为 5×200 ms $=1$ s, 平均速率为 $20\times1024\times8/1=$ **163840 bit/s** $=$ **163.84 kb/s**。这里 KB 用 1024、kb/s 用 1000；计时起点不包括握手，但终点包括最后一轮确认回传。

零窗口是接收方流量控制的结果，并不说明此前有数据丢失，也不要求按超时规则把 cwnd 重置为 1。发送方可用持续计时器触发零窗口探测，以免窗口更新丢失后永久等待；探测本身不会使接收缓存腾空。在本题 “始终不取数据” 的条件下，不能假设经过一个 RTT 窗口便自动恢复。

第（4）问求 S 的释放时间。为取最短值，S 收到 H3 的 FIN 后立即合并发出 ACK 和自己的 FIN：H3 的 FIN 到 S 用 0.5 RTT，S 的 FIN 到 H3 再用 0.5 RTT，H3 的最后 ACK 返回 S 再用 0.5 RTT。S 收到此 ACK 后进入 CLOSED，故最短为 1.5RTT $=$ **300 ms**。H3 作为主动关闭方的 TIME-WAIT 等待属于另一端的计时，不能再加到 S 的 300 ms 上。

应用层

例 6-1 C

计数主体是 201.1.1.1，不是 H4。最终地址在有效缓存中时，本地服务器不需向外查询，最少 0 次；完全无可用缓存，且相应各级分别委派时，可依次询问根、com、xyz.com、abc.xyz.com，共 4 次，选 **C**。H4 向 201.1.1.1 的最初请求不是这台服务器 “发出” 的请求，不能加入。

“最多 4 次” 采用原题的规范域名及逐级委派模型，不计别名链、失败重试或负载均衡额外查询。若改问主机发出几条请求，或所有层级均以递归方式向下转交，通信主体与计数就要重新列出，而不是照抄 0、4。

例 6-2 A

13. A

对于 FTP 文件传输，为了保证可靠性，选择 TCP，排除 C 和 D。FTP 的控制信息是带外传送的，即 FTP 使用了一个分离的控制连接来传送命令，因此答案为选项 A。

例 6-3 A

12. A

POP3 用于用户代理从邮件服务器读取邮件，说法 I 正确。用户代理向邮件服务器发送邮件使用的是 SMTP，而非 POP3。邮件服务器之间的邮件传输也由 SMTP 实现，与 POP3 无关。POP3 支持在一条 TCP 连接中收取多封邮件，无须为每封邮件单独建立连接，说法 IV 正确。

例 6-4 D

19. D

浏览器不支持并行 TCP 连接，使用非持续的 HTTP/1.0 协议，因此每传输一个 Web 页和小图像文件都要建立一次 TCP 连接。第一次建立 TCP 连接时，前两次握手花 1RTT，第三次握手报文段中可以携带 HTTP 请求，服务器收到请求后返回 Web 页，共花 2RTT。之后，每传输一个图像文件都要花 2RTT。因此，到接收完所有内容，需要的总时间至少是 2×8 = 16RTT。注意，若浏览器支持并行 TCP 连接，则请求 Web 页仍要花 2RTT，但收到 Web 页后，可建立 7 个并行的 TCP 连接请求图像文件，传输图像的过程仅花 2RTT，总时间为 4RTT。

例 6-5 A

12. A

从发出 HTTP 请求报文开始，所以此时 TCP 连接已经建立。本题采用了流水线的持续连接。第 1 个 RTT 请求并收到 html 页面，收到 html 页面后才能发送对其引用小图像的请求，所以第 2 个 RTT 请求并收到 3 幅小图像，合计耗费 2RTT。

例 6-6 D

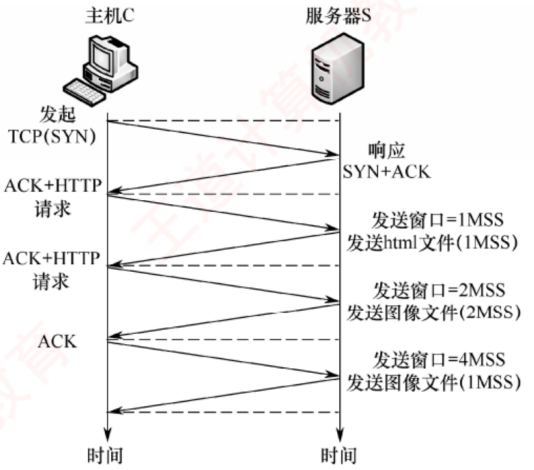
采用题目首次取页面、尚未建立目标 TCP 连接的模型。有效 DNS 缓存命中时，不需要远程 DNS 往返，但 TCP 建立与 HTTP 请求响应各 1RTT，最短 20 ms。完全无所需缓存时，本地解析器依次询问根、com、abc.com 权威服务器，增加 3RTT；再取 Web 共 5RTT=50 ms，选 **D**。主机与本地 DNS 之间的局域网时延已忽略，不能额外加一次 10 ms。

如果题目再允许浏览器已缓存新鲜的 HTML 副本，可能无须访问服务器；若 TCP 已经建立，也会少一个握手往返。这些是改变原题模型的附加条件，不能在原题中任意引入后声称存在另一个更短答案。

例 6-7 B

18. B

HTTP/1.1 默认使用持续连接，所有请求都是连续发送的。要求最少时间，理想的情况是 TCP 在第 3 次握手的报文段中捎带了 HTTP 请求，以及传输过程中的慢开始阶段不考虑拥塞。假设接收端有足够大的缓存空间，即发送窗口等同于拥塞窗口，共需要经过：第 1 个 RTT，进行 TCP 连接建立的前两次握手；第 2 个 RTT，主机 C 发送第 3 次握手报文并捎带了对 html 文件的 HTTP 请求，TCP 连接刚建立时服务器 S 的发送窗口 = 1MSS，服务器 S 发送大小为 1MSS 的 html 文件；第 3 个 RTT，主机 C 发送对 html 文件的确认并捎带了对图形文件的 HTTP 请求，服务器 S 收到确认后发送窗口变为 2MSS，然后服务器 S 发送大小为 2MSS 的图像文件；第 4 个 RTT，主机 C 向服务器 S 发送对收到的部分图像文件的确认，服务器 S 收到确认后发送窗口变为 4MSS，然后服务器 S 发送剩下的 1MSS 图像文件，完成传输，共需要 4RTT，即 40ms。整个传输过程如下图所示。



例 6-8 （1）控制持久、数据非持久，先建控制；（2）101、18102；（3）3 MSS、5 MSS；（4）60 ms、2.4 Mb/s。

先把 FTP 会话与一次文件传输分开。控制连接在整个 FTP 会话期间保持，承载登录、命令与控制信息，因此是**持久连接**；本题的数据连接服务于本次文件传输，传完后关闭，是**非持久连接**。登录时先建立的是**控制连接**，已经完成的登录过程不再计入第（4）问的数据连接计时。

数据连接的 SYN 占序号 100，F 首字节从 **101** 开始。18000 B 文件占序号 101～18100，发送完后 H 的 FIN 占 18101，服务器第二次挥手的 ACK 应确认下一个期望序号，故为 **18102**。这里确认的是文件数据再加 FIN，不能把登录控制连接的序号混进来。

拥塞控制沿用教材的轮次模型：初始 $cwnd = 1$ MSS，达到 4 MSS 门限后按每 RTT 增加 1 MSS。先看 ACK 落在哪个时刻：确认号 2101 累计确认 $2101 - 101 = 2000$ B，即前 2 个段。第 1 个 ACK 使 cwnd 从 1 增为 2 MSS；第 2 个 ACK 是第二轮中较早的那个，此时仍在慢开始，cwnd 从 2 增为 **3 MSS**。不能把 “第二轮结束时 4 MSS” 提前用于这个 ACK。

确认号 7101 累计确认 $7101 - 101 = 7000$ B，即 $1 + 2 + 4 = 7$ 个段，恰好完成第 3 轮。此轮开始时已到 4 MSS 门限，采用拥塞避免，第 3 轮结束后窗口为 **5 MSS**。下表同时列出数据发送、确认时刻与累计量，时间均从数据连接 SYN 发出算起。

阶段	开始时刻/ms	本轮 cwnd/MSS	本轮发送段数
数据连接握手	0	—	—
数据第 1 轮	10	1	1
数据第 2 轮	20	2	2
数据第 3 轮	30	4	4
数据第 4 轮	40	5	5
数据第 5 轮	50	6	6

阶段	累计发送段数	本轮全部确认到达/ms
数据连接握手	0	10
数据第 1 轮	1	20
数据第 2 轮	3	30
数据第 3 轮	7	40
数据第 4 轮	12	50
数据第 5 轮	18	60

建立数据连接用 1 RTT，18 个段按 1、2、4、5、6 分 5 轮发完，并在最后一轮结束时得到全文件确认，所以总计 6RTT = **60 ms**。期间应用层平均速率为 $18000 \times 8 / 0.060 = \mathbf{2.4\ Mb/s}$，也就是 300000 B/s=0.3 MB/s。分母按题目包含握手、不包含连接释放；不能只用 50 ms 的数据传输阶段，也不能因服务器较早收到最后一比特就省去最后确认的回程。

这里的按轮增长是题目采用的教材近似，阈值前逐 ACK 检查仍有意义；拥塞避免阶段不宜把轮次表误解成每个 ACK 都使 cwnd 增加整整 1 MSS。若要描述某个操作系统的精确逐 ACK 轨迹，还需其拥塞算法和 ACK 策略。

例 6-9 （1）DNS，UDP/IP/以太网封装；（2）H1→4、DNS→1、R→2；（3）2 帧，目的全 FF（需新 DNS 查询）。

解题入口是按依赖关系列出 “先知道谁的地址，才能发下一帧”。按本题及教材需要新做域名解析的流程，H1 先运行 **DNS**，把域名换成 Web 服务器 IP；常规查询的封装为 **DNS 报文 →UDP 数据报 →IPv4 分组 →Ethernet II 帧**。ARP 用于在本地链路上解析 MAC，不是 DNS 外面的一层，也不是另一个应用层协议；CSMA/CD 是介质访问算法，不是帧格式，在全双工交换链路中通常不运行。

H1 先根据 /25 判断本地 DNS 的 192.168.1.126 与自己同属 192.168.1.0/25，先用 ARP 解析本地 DNS 的 MAC 地址，不先问网关。解析出 Web 服务器位于外网后，才 ARP 查询网关 R 的 192.168.1.1。后续 IP 分组的最终目的仍是 Web 服务器，当前以太网帧的目的 MAC 却是 R 的 MAC。

顺序	S 收到的帧	入端口	S 的处理与新增学习
1	H1 广播 ARP，询问本地 DNS 的 MAC	4	学习 H1 的 cc→4；向 1、2、3 口泛洪
2	DNS 单播 ARP 应答给 H1	1	学习 DNS 的 bb→1；已知 cc，只转 4 口
3	H1 单播 DNS 查询给 DNS	4	已知 bb，只转 1 口
4	DNS 单播查询应答给 H1	1	已知 cc，只转 4 口
5	H1 广播 ARP，询问网关 R 的 MAC	4	向 1、2、3 口泛洪
6	R 单播 ARP 应答给 H1	2	学习 R 的 aa→2；只转 4 口
7	H1 向外网服务器发送 TCP SYN，帧发给 R	4	已知 aa，只转 2 口
8	R 转发服务器的 SYN+ACK，帧发给 H1	2	已知 cc，只转 4 口
9	H1 发送 TCP 确认及首个 HTTP 请求	4	帧均发给 R，只转 2 口；到达题设 t_1

表中 cc、bb、aa 分别是下表完整 MAC 地址的末字节缩写。第三次握手 ACK 是否与首个 HTTP 请求合并，不改变学习或 H2 收到的帧数。交换机按**帧的源 MAC 与入端口**学习，不会从广播帧的目的地址、ARP 询问目标、IP 目的地址或题图上 “知道有 H2” 便自行添加表项。因此 t_1 时为：

MAC 地址	端口
00-11-22-33-44-cc（H1）	4
00-11-22-33-44-bb（本地 DNS）	1
00-11-22-33-44-aa（R）	2

H2 只被动收到两次广播 ARP 请求，没有发出自己的帧，故没有 dd→3 条目。两次单播应答到来时，交换机早已从相应请求学到 H1 的位置，不会因初始表为空便把所有单播一直泛洪。第（3）问按上述解析流程，H2 至少收到 **2 个 ARP 请求帧**，分别查询本地 DNS 和网关，目的 MAC 均为 **FF-FF-FF-FF-FF-FF**。

这个 2 帧结论以本题意图中的新 DNS 查询为前提。原题明确为空的是 ARP 表和交换表，并未额外声明 DNS 缓存为空；若允许命中已有 DNS 缓存，就不必先访问本地 DNS，相应学习表与最小广播数都会改变。应保留这一条件边界，不能把 “通过域名访问” 推广成任何情况下都必然重新发送 DNS 查询。